

TECHNICAL UNIVERSITY - SOFIA
Faculty of Computer Systems and Technologies



Bachelor's Thesis

**MULTIMODAL PROJECTION MAPPING-BASED
3D OBJECT INSPECTION SYSTEM USING
GESTURE AND VOICE INTERACTION**

Author: Yana Najy

Faculty Number:273222007

Email Address: ynazhi@tu-sofia.bg

Supervisor: Prof. PhD Milena Lazarova

2026

TABLE OF CONTENTS

<i>ABSTRACT.....</i>	<i>4</i>
<i>I. PROBLEM DEFINITION AND THESIS AIM.....</i>	<i>6</i>
1.1 Thesis Aim.....	9
1.2 System Scope and Functionalities	9
1.3 limitations.....	10
1.4 Comparing with Existing Multimodal Interaction Systems	11
<i>II. THEORETICAL BACKGROUND AND LITERATURE REVIEW</i>	<i>13</i>
2.1 Gesture recognition	13
2.2 Voice Recognition	14
2.3 Communication.....	15
2.4 Projection-based interaction.....	17
2.5 Technologies	18
<i>III. SYSTEM DESIGN AND IMPLEMENTATION.....</i>	<i>21</i>
3.1 System Architecture	21
3.2 Hardware and Software Environment	22
3.3 Unity Environment Development.....	24
3.4 Gesture Recognition System	25
3.5 Voice Recognition System	27
3.6 Object Inspection and Interaction System	29

3.7 Projection-Based Visualization	32
<i>IV. USER MANUAL AND ANALYSES OF EXPERIMENTAL RESULTS.....</i>	<i>35</i>
4.1 User Manual.....	35
4.2 Experimental Setup	42
4.3 Discussion of Results	50
<i>CONCLUSION</i>	<i>52</i>
<i>REFERENCES.....</i>	<i>54</i>
<i>APPENDIX.....</i>	<i>58</i>
ObjectControl.cs.....	58
Gesture.py	66
Voice.py	69

ABSTRACT

The increasing demand for intuitive Human-Computer Interaction (HCI) has encouraged the development of systems that enable users to interact with three-dimensional objects through natural communication methods rather than conventional input devices. Conventional object inspection systems may restrict accessibility and user involvement since they frequently depend on a mouse, keyboard, or specific wearable devices. This study proposes a multimodal projection mapping-based 3D object inspection system that integrates gesture recognition, voice recognition, and projection-based visualization into a unified interactive environment.

Python was used for gesture and voice processing, and Unity was used for three-dimensional visualization in the system's implementation. The SpeechRecognition library was used for voice command processing, and the MediaPipe framework was used for real-time hand landmark detection and gesture recognition. Communication between the Python subsystem and the Unity application was established through UDP socket communication, enabling the real-time transmission of interaction commands.

The system allows users to manipulate a projected 3D robot model using predefined gestures for object rotation, camera zoom, and scene reset, while voice commands enable component-focused inspection. When a valid component is selected, the system automatically focuses the camera on the corresponding region, applied visual highlighting, displays the appropriate label and information panel, and updates the status indicator to provide immediate feedback to the user. In order to create an interactive projection-based inspection environment without the need for wearable devices or specialized tracking equipment, the final visualization is projected onto a physical surface.

Experimental evaluation demonstrated that the developed system successfully integrated gesture recognition, voice recognition, and projection-based visualization into a functional multimodal interaction platform. Iterative improvements led to reliable system performance under normal operating conditions, although environmental conditions including lighting, background noise, and projector characteristics affected overall responsiveness. The study further demonstrates that effective multimodal interaction can be achieved using affordable and widely available hardware while maintaining an intuitive and user-friendly inspection experience.

Beyond the implemented prototype, the suggested approach may find use in medical visualization, engineering design, robotics education, architecture and real estate presentation, and industrial training, where interactive three-dimensional inspection can improve analysis, communication, and decision-making.

Keywords: Human-Computer Interaction, Multimodal Interaction, Gesture Recognition, Voice recognition, Projection Mapping, 3D object inspection, Unity, MediaPipe.

I. PROBLEM DEFINITION AND THESIS AIM

The focus of today's developments in Human-Computer Interaction (HCI) have shifted significantly from focusing on traditional input methods towards more natural and immersive interaction techniques. Keyboards, mice, and touch interfaces are conventional interaction approaches that are insufficient for complex three-dimensional environments and real-time object manipulation tasks. As interactive visualizations become increasingly common in fields such as education, engineering, robotics, and simulation, researchers have explored alternative interaction paradigms that improve usability, accessibility, and interaction efficiency.

Multimodal interaction is one of the primary fields of HCI research. In order to produce more natural interaction systems, multimodal human-computer interaction incorporates various communication modalities, namely gestures, speech, touch, and visual input. By allowing users to engage through numerous simultaneous channels instead of depending on a single input modality, multimodal interaction systems seek to mimic genuine human communication [2]. The study emphasizes, by dispersing interaction activities among complementary modalities, multimodal interfaces increase flexibility, decrease ambiguity, and improve user experience [2]. This concept forms the theoretical foundation of the proposed system developed in this thesis.

One of the most extensively researched types of natural interaction is gesture recognition systems. Gesture-based recognition enables users to manipulate digital objects through hand and body gestures without requiring physical controllers. In applications involving robotics, augmented reality, virtual environments, and three-dimensional visualization systems, gesture interaction has grown in significance. Gesture-based human-machine interaction systems, translate natural hand movements into machine-understandable commands in order to provide more intuitive and effective communication between humans and digital systems [1]. Their research classifies gesture interaction techniques and emphasizes the significance of usability and real-time responsiveness in modern HCI systems [1].

The performance and usability of gesture recognition systems have been greatly enhanced by recent developments in computer vision. Earlier gesture interaction systems frequently required specialized hardware, such as depth cameras or sensor gloves, increasing implementation complexity and cost. However, in order to perform real-time gesture detection, modern approaches combine RGB cameras with machine learning and landmark tracking techniques. Recent research

on visual gesture recognition has demonstrated that computer vision framework can achieve high recognition accuracy while operating with low-cost consumer hardware [6]. The practicality of gesture-based interfaces for interactive real-time applications has improved as a result of this advancement.

Real-time performance is particularly important for interactive object manipulation systems. To preserve interaction fluidity and usability, gesture recognition systems used in immersive environments must react to user movements fast and precisely. According to *Mohamed (2024)*, modern real-time hand gesture recognition systems heavily focus on minimizing latency, improving tracking robustness, and supporting natural interaction in dynamic environments [3]. For systems that perform continuous manipulation tasks such as object rotation, scaling, and navigation, these characteristics are essential.

In immersive visualization and analytics systems, three-dimensional gestural interfaces are becoming increasingly important. Studies explain, gesture-based interaction in 3D environments enhances user immersion and spatial comprehension by enabling users to engage with virtual objects through natural physical gestures. Furthermore, it elaborates spatial interaction techniques as particularly effective for object inspection tasks because they provide users with direct control over object orientation and visualization [5]. These findings are compelling evidence in favor of the suggested 3D object inspection system's usage of gesture interaction.

Despite their advantages, gesture-only systems may encounter drawbacks related to precision, fatigue, and discrete command selection. Continuous gesture interaction can become physically demanding during extended use, and gesture-only systems may struggle when users need to select specific object components or activate particular system nodes. Researchers are progressively integrating voice interaction methods with gesture recognition in multimodal systems to overcome these constraints.

Hands-free communication and discrete command execution are made possible by voice interaction systems. With speech-based interfaces, users may naturally issue commands without requiring additional physical interaction. Research claims that because each modality can be assigned tasks that play to its strengths, multimodal systems that integrate gesture and spoken interaction increase interaction efficiency [2]. While gestures are better suited for continuous spatial manipulation activities, voice interactions especially useful for symbolic or discrete actions like object selection, mode switching, and command activation.

A central topic within multimodal HCI research is the integration of gesture and voice interaction. To minimize uncertainty and enhance usability, modern multimodal systems frequently divide interaction responsibilities across different modalities. Within the proposed system in this thesis, gesture recognition is used for continuous manipulation tasks such as object rotation, zooming, and scene reset, while voice commands are used for selecting specific robot components including the head, arms, and legs. By allocating interaction tasks based on modality strength, this interaction structure adheres to modern multimodal interaction concepts.

In addition to interaction methods, immersion inspection systems heavily rely on visualization technologies. Another significant field of study pertaining to immersive interaction environments is projection-based and spatial visualization systems. Systems for spatial augmented reality and projection mapping build shared interactive places by fusing digital depiction with real-world settings. *Printezis (2026)* claims that by integrating digital data directly into real-world settings, projection-based visualization technologies improve accessibility and immersion [4]. Unlike fully immersive virtual reality systems that require head-mounted displays, projection-based systems facilitate teamwork and reduce user discomfort associated with wearable devices [4].

Furthermore, gesture interaction has also become increasingly important within robotic and object-inspection environments. According to *Wang et al. (2022)*, gesture-based human-robot interaction systems facilitate intuitive user-robot interaction, which increases communication effectiveness and user engagement [7]. According to their review, hand and arm gestures are particularly effective for navigation, manipulation, and inspection tasks involving robotic models and spatial interfaces [7]. These findings further support the implementation approach adopted in this thesis.

When compared to standard interfaces, multimodal interaction systems offer notable benefits in terms of usability, immersion, adaptability, and interaction efficiency, according to the studied research. Incorporating gesture recognition, voice interaction, and projection-based visualization into a real-time three-dimensional inspection environment offers a solid theoretical basis for the implementation developed in this thesis and is a practical application of current HCI research trends.

1.1 Thesis Aim

The goal of the thesis is to design and develop multimodal projection mapping-based 3D object inspection system using gesture and voice interaction. Through hand gestures and voice commands users can interact in real time with a projected three-dimensional robot model using the system. Gesture interaction is used for object manipulation functions such as rotation, zooming, and scene reset, while voice commands are used for component-focused inspection actions including selecting specific robot body parts like head, right arm, and left leg. The system creates a unified interactive environment by integrating MediaPipe-based gesture recognition, speech recognition, UDP socket communication, and Unity-based 3D visualization. To enhance user engagement and interaction clarity, the system additionally incorporates object highlighting, label visualization, camera focus control, and projection-based display output. The implementation uses low-cost consumer hardware and real-time visualization technologies to provide an accessible and intuitive multimodal interaction environment that is both accessible and intuitive.

1.2 System Scope and Functionalities

The goal of the system is to provide a 3D object inspection environment based on multimodal projection mapping that integrates voice interaction, gesture recognition, and projection-based visualization into a unified Human-Computer Interaction framework. Integrating multiple interaction modalities enhances interaction flexibility, usability, and user experience in immersive digital environments, according to recent research in multimodal HCI [8].

The developed system supports several real-time interaction functionalities. Continuous object manipulation tasks, such as object rotation, zooming, and scene reset, rely on gesture recognition. For discrete interaction commands, such as selecting specific robot body parts like the head, arms and legs, voice recognition is utilized. In accordance with modern multimodal interaction concepts, this modality separation assigns symbolic orders to speech interaction and continuous tasks to gesture interaction [26].

The system additionally supports camera focus control, label visualization, object highlighting, and projection-based visual feedback. The three-dimensional inspection environment is rendered and managed by Unity, while gesture and voice input are processed by Python using the MediaPipe and SpeechRecognition frameworks. UDP socket communication is used to

maintain low-latency interaction performance between the processing subsystem and the visualization environment.

Low-cost consumer hardware, such as webcam, microphone, projector, and personal computer, is intended to run the implementation. Unlike immersive systems requiring head-mounted displays or specialized tracking hardware, the proposed system emphasizes accessibility, simplicity, and real-time interaction within a projection-based visualization environment.

1.3 limitations

The system is implemented as a prototype multimodal interaction environment designed for interaction research and real-time object inspection. Although the system supports interactive gesture and voice control, there are several limitations within the current implementation.

Currently, the system allows single-user interaction and relies on predefined gesture patterns and predefined voice commands for interaction control. Users are limited to performing gestures using only one hand at a time, as the gesture recognition module was designed for single-hand interaction. The simultaneous use of both hands may interfere with hand landmark detection and reduce recognition accuracy. Environmental factors like lighting variations, camera positioning, hand occlusion, and background complexity may influence gesture recognition performance. Similarly, speech clarity, pronunciation variations, ambient noise, and microphone quality can all have an impact on voice recognition accuracy. Research on real-time multimodal interaction systems identifies major drawbacks of vision-based and speech-based interaction systems is environmental sensitivity [9].

Furthermore, rather than sophisticated industrial projection mapping calibration techniques or spatial augmented reality tracking, the projection setup concentrates on fundamental projection alignment and visualization. The system also ensures low hardware requirements and real-time responsiveness, through lightweight gesture classification methods rather than computationally demanding deep learning models.

The current thesis implementation does not include advanced capabilities like dynamic gesture detection, natural language processing, multi-user interaction, adaptive machine learning optimization, and full spatial environment calibration. Notwithstanding these drawbacks, the created system effectively illustrates how multimodal interaction approaches may be integrated into a projection-based 3D inspection environment.

1.4 Comparing with Existing Multimodal Interaction Systems

Multimodal interaction systems that integrate voice and gesture input for immersive environments have been the subject of recent research. However, the majority of existing systems focus on specific application domains like robotic interactions, augmented reality, or virtual reality.

For object manipulation in augmented reality environments, *Zhou et al. (2022)* developed a multimodal gesture-and-speech interaction framework. Their approach used augmented reality headsets to examine user-defined speech and gesture combination for virtual object interaction. The study demonstrated that during object manipulation tasks, combining gesture and voice interaction increased interaction efficiency and decreased ambiguity. However, the implementation primarily focused on augmented reality environments requiring specialized wearable hardware [29].

In a similar vein, *Wang et al. (2025)* examined current multimodal natural interaction methods applied to Extended Reality (XR) systems. The growing usage of hand gestures, voice commands, gaze interaction, and spatial computing approaches for immersive environments was emphasized on by their analysis. Despite providing sophisticated interaction features, these systems commonly rely on sophisticated tracking technologies and wearable XR headsets [27].

Recent robotic interaction research has also investigated multimodal gesture and voice control. An augmented reality-based robot teleoperation system that integrates voice commands and hand gestures for robot control was introduced by *Zhang et al. (2025)*. According to their findings, they indicated that multimodal interaction outperformed gesture-only interaction in terms of usability and task performance. However, the system's reliance on extensive language model integration and augmented reality hardware increased the complexity of its implementation [28].

In contrast, the system proposed in the thesis combines gesture recognition, voice interaction, projection-based visualization, and real-time 3D object inspection within a unified environment using low-cost consumer hardware. The proposed implementation uses a webcam, microphone, projector, and personal computer to provide natural multimodal interaction without the need for wearable technology, in contrast to many contemporary immersive systems that rely on augmented reality or virtual reality headsets. The system additionally integrates a combination

of projection-based interaction, label visualization, camera focus control, and component-focused object inspection into a single real-time application.

SYSTEM REFERENCE	Gesture Interaction	Voice Interaction	Projection- Based Visualization	Wearable Hardware Required	Object Inspection Features
AR-Based Maintenance Inspection (2022)	Yes	Yes	No	Yes	Limited
XR Training Platform (2023)	Yes	Yes	No	Yes	Moderate
Robot teleoperation Teleportation (2024)	Yes	Yes	No	Yes	Moderate
Implemented System	Yes	Yes	Yes	No	Yes

Table 1 Comparison of Existing Systems and Proposed System

II. THEORETICAL BACKGROUND AND LITERATURE REVIEW

Several software technologies interaction methodologies must be integrated in order to construct the multimodal projection mapping-based 3D object inspection system. Gesture recognition, voice recognition, real-time communication, and projection-based visualization are combined to create an interactive inspection environment that supports natural user interaction. Combining multiple interaction modalities improves interaction flexibility, usability, and user experience in immersive environments, according to recent research in multimodal human-computer interaction [8].

This chapter focuses on the methodologies used for gesture processing, speech-based interaction, communication between subsystems, and projection-based interaction. Furthermore, the chapter provides an overview of the main technologies utilized in the implementation, such as TextMeshPro, Unity Engine, Python, MediaPipe, SpeechRecognition, and UDP socket communication.

2.1 Gesture recognition

Gesture recognition enables users to interact with digital environments through natural hand movements instead of traditional input devices, it is a crucial component of modern multimodal HCI systems. Gesture-based interaction enhances intuitiveness and facilitates direct object manipulation in immersive three-dimensional environments. Recent developments in computer vision have significantly improved the accessibility and performance of real-time gesture recognition systems using standard RGB cameras [9].

This thesis utilizes a vision-based hand tracking methodology using a webcam input. Image capture, hand landmark detection, gesture classification, and command interpretation are what the gesture recognition process comprises of. Modern landmark-based tracking methods are extensively used because of their ability to reduce computational complexity while maintain accurate real-time interaction performance [10].

In order to identify and track hand landmarks in real time. The built system makes use of MediaPipe hand tracking. The methodology tracks predefined landmark points that correlate to finger joints and palm positions rather than analyzing the entire hand image. These landmarks are then used to determine gesture state through geometric calculations such as finger extension analysis and distance measurements between landmarks.

The proposed system uses rule-based analysis of landmark placements to classify gestures. Pinch gestures are used for zooming operations, open-hand gestures are mapped to object rotation, and closed-hand gestures are associated with reset. In the 3D inspection environment, this lightweight classification approach reduces latency and supports responsive engagement.

An essential prerequisite for immersive interaction systems is real-time responsiveness. Research on gesture recognition methodologies emphasizes that low computational overhead and tracking stability are essential for maintaining smooth user interaction [11]. Because of this, the proposed methodology places more emphasis on streamlined gesture detection logic and effective landmark tracking than on computationally demanding deep learning classification models.

Although vision-based gesture recognition systems offer intuitive and natural interaction, identification accuracy may be impacted by ambient factors including lighting, background complexity, and camera placement. However, because of its cheap hardware requirements and ease of integration, webcam-based gesture interaction is still a viable and accessible approach for real-time multimodal inspection systems.

In order to facilitate user-friendly interaction inside the developed projection-based 3D object inspection system, the suggested gesture detection approach integrates lightweight gesture classification techniques with real-time hand landmark tracking.

2.2 Voice Recognition

Voice recognition is used in this project as a complementary input modality to support hands-free interaction with the 3D inspection system. Voice commands work better for discrete activities like choosing particular object sections, resetting the scene, or activating inspection modes, whereas gesture recognition works better for continuous actions like rotation and zooming. According to recent studies, speech recognition systems are becoming widely popular in HCI because they enable users to give natural commands without relying on physical input devices [13]. Voice-based interaction is particularly effective for discrete actions such as object selection, mode switching, and command activation.

A command-based speech recognition approach is utilized in the voice recognition methodology. A microphone is used to capture audio input, which is then processed using speech-to-text conversion. A predetermined list of terms, including “head”, “left arm”, “right leg”, “zoom”,

and “reset”, is then compared to the recognized text. When a matching keyword is detected, the system converts it into a control command and sends it to the Unity environment.

Because constrained command vocabularies increase recognition reliability and lower ambiguity in real-time systems, this paradigm was chosen. Simplified command structures are advantageous for voice recognition systems intended for human-computer interaction since they increase processing speed and lower recognition mistakes in real-world scenarios [14]. Voice interaction also enables more natural communication between users and digital systems by minimizing the need for physical input devices.

Component-focused inspection duties are mainly handled by voice commands in the suggested system. For example, when the user says “head”, the camera focuses on the robot’s head while the corresponding label is displayed. This interaction structure separates discrete selection tasks from continuous manipulation tasks, which are handled through gesture recognition. In addition to improving interaction clarity and usefulness, this kind of modality separation is consistent with contemporary multimodal interaction principles.

Even though voice recognition technologies make communication easier and more accessible, factors including background noise, microphone quality, pronunciation variations, and speech clarity may affect how well they work. In order to preserve consistent recognition performance during real-time interaction, the adopted methodology makes use to brief and easily identifiable commands.

Overall, the suggested voice recognition methodology supports the project’s larger multimodal interaction framework while offering a lightweight and practical solution for integrating speech-based interaction into the projection-based 3D object inspection system.

2.3 Communication

UDP socket communication is used to enable real-time communication between the Unity visualization environment, the gesture recognition subsystem, and the voice recognition subsystem. For responsive interaction and real-time feedback in multimodal interactive systems, efficient communication mechanisms are crucial for transmitting user commands with minimal latency. Recent Research on interactive simulation and visualization systems highlights the importance of lightweight communication protocols for real-time synchronization between external processing applications and Unity-based environments [16].

This thesis implements a client-server communication paradigm with User Datagram Protocol (UDP) as its communication methodology. Unity acts as the visualization and interaction environment and interaction environment, while the Python subsystem handles voice and gesture processing. Once a gesture or voice command is recognized, the UDP socket acts as a bridge between the python application which transmits a short command to the Unity application running on a predefined local port.

UDP was selected because of its lightweight structure and low communication overhead compared to a Transmission Control Protocol (TCP) which is a connection-oriented protocol. According to research on real-time communication systems, UDP is well suited in interactive settings that need fast transmission speeds and low latency [15]. UDP is an effective choice for the project because the suggested system only sends small command messages like “ROTATE”, “ZOOM”, and “HEAD”, negating the need for dependable packet recovery procedures.

In contrast, TCP establishes a connection before data transmission and incorporates mechanism such as packet acknowledgment, retransmission, error checking, and flow control to guarantee reliable delivery. Although these features improve communication reliability, they introduce additional latency and processing overhead that are unnecessary for the proposed real-time interaction system. Transmission isn’t a priority as it prioritizes maintaining continuous and responsive interaction, meaning that if a command is delayed or lost, the subsequent gesture or voice command generated by the user quickly replaces it. Therefore, UDP provides a more suitable communication protocol for the developed multimodal inspection system.

Within Unity, a socket receiver keeps an eye out for incoming UDP communications and translates them into interaction commands. This communication workflow allows for independent and real time synchronization for gesture recognition, voice recognition, and visualization modules. Similar communication architectures have been widely adopted in simulation and visualization systems that integrate Unity with external processing environments such as Python and C++ applications [16].

Additionally, the communication mechanism enhances the system architecture’s modularity. By separating processing tasks from visualization tasks, the system is easier to maintain, debug, and expand. For instance, the Unity visualization environment does not need to be altered in order to modify gesture recognition algorithms. This modular structure aligns with current real-time interactive system design approaches.

Although UDP communication provides low-latency performance, but because packet delivery is not guaranteed, packet loss may occasionally occur. However, as interaction commands are constantly created and updated during runtime, this constraint has little effect on the proposed system. All things considered, the adopted communication methodology offers a lightweight and efficient solution to enable multimodal interaction in real time within the projection-based 3D object inspection system.

2.4 Projection-based interaction

The proposed system utilizes a projection-based interaction methodology to display and manipulate three-dimensional content within a physical environment. Because they enable collaborative and immersive interaction without the need for wearable devices or head-mounted displays, projection-based visualization systems are becoming increasingly popular in HCI. Projection mapping is a method that creates interactive environments by spatially aligning digital content onto physical surfaces, according to recent research on spatial augmented reality [19].

In this thesis, the Unity-generated 3D scene is projected onto a wall surface using a projector-based visualization technique. The projected environment acts as the primary interaction space for object inspection and manipulation. Using speech and gesture commands, users can interact with the projected 3D model in real time without making direct physical contact. This interaction structure supports intuitive and natural Human-Computer Interaction principles.

Because projection-based interaction reduces the physical constraints associated with wearable immersive technologies and facilitates collaborative observation and shared visualization, it was selected. Research on immersive interaction environments indicates that by embedding digital information directly into surrounding physical space, accessibility and user comfort improve in projection-based systems [20].

In the proposed system, Unity manages real-time rendering and interaction updates, and the projector acts as the visualization output device. The unity environment receives user commands generated through gesture and voice commands recognition, and the projected 3D object responds by rotating, zooming, and performing component focused inspection actions. The user, processing subsystem, and projected visualization all engage in a constant loop of communication as a result.

To guarantee correct alignment and visibility of the projected scene, basic projection calibration was carried out. To enhance interaction clarity and visualization quality, calibration

comprised projector position, image scaling, and perspective adjustment. The suggested implementation focuses on accessible projection-based interaction appropriate for real-time multimodal object inspection, in contrast to sophisticated industrial projection mapping systems.

In general, the projection-based interaction methodology creates an immersive and user-friendly 3D object inspection environment by combining multimodal control, spatial interaction techniques, and real-time visualization.

2.5 Technologies

2.5.1 Unity Engine

A popular real-time 3D development tool for interactive visualization, simulation, and HCI applications is Unity Engine. It provides C# scripting capabilities for scene building, object rendering, camera control, and real-time interaction [21].

The interactive 3D inspection environment in this project is created and displayed using Unity. The robot model's visualization, object rotation and zooming, camera movement, label display, and real-time gesture and voice command response are all handled by the engine. Unity was chosen due to its capability for interactive systems, real-time rendering capabilities, and compatibility with external technologies like Python and UDP socket communication.

2.5.2 Python

Due to its ease of use, flexibility and extensive library support, Python is a high-level programming language that is frequently utilized in applications related to machine learning, computer vision, automation, and human-computer interaction [22]. It commonly uses real-time processing systems because it enables rapid development and integration with external frameworks and communication protocols.

The gesture and voice recognition subsystems in this project are implemented using Python. Libraries such as MediaPipe, OpenCV, and SpeechRecognition are used within Python to process webcam and microphone input in real time. Additionally, Python is in charge of sending interaction commands to Unity through UDP socket communication. The language was selected because of its compatibility with computer vision and speech processing frameworks, as well as its ability to support real-time interaction workflows efficiently.

2.5.3 MediaPipe

MediaPipe is an open-source framework developed by Google for real-time machine learning and computer vision applications. Using lightweight landmark detection models that work with ordinary RGB cameras, it provides efficient hand, face, and body tracking solutions [23].

In this project, MediaPipe is utilized for gesture recognition and real-time hand tracking. The system can recognize motions like open hand, pinches, and fist gestures thanks to the framework's detection of hand landmarks and tracking of finger positions. Because MediaPipe offers precise real-time tracking, low computational overhead, and easy integration with Python-based computer vision systems, it was chosen.

2.5.4 SpeechRecognition Library

The SpeechRecognition library is a Python-based voice processing library that uses speech-to-text recognition technique to translate spoken words into text. It is appropriate for Human-Computer Interaction applications involving voice-controlled devices since it allows real-time microphone input and integration with several speech recognition engines [24].

In order to operate the 3D inspection environment, voice commands are recorded and processed using the SpeechRecognition library. Within Unity, recognized commands like “head”, “left arm”, and “reset” are converted into interaction actions. The library was selected because it provides simple implementation, real-time voice processing capabilities, and easy integration with Python-based interaction systems.

2.5.5 UDP socket communication

A lightweight networking technique called UDP socket communication is used to send data in real time between applications. Unlike connection-oriented protocols such as TCP, UDP provides faster communication with lower latency, making it suitable for interactive systems that require rapid data transfer [25].

In this project, voice and gesture commands are sent from the Python processing subsystem to the Unity visualization environment using UDP socket communication. When a command is identified, Python transmits a short UDP message, to Unity where the relevant interaction is carried out instantly. It is a great channel because UDP communication offers an effective low-latency performance and straightforward integration between Python and Unity applications.

2.5.6 TextMeshPro

Unity uses TextMeshPro, a text rendering system, to display customized, high-quality text in dynamic real-time environments. Compared to Unity's standard text system, TextMeshPro provides improved text clarity, advanced styling options, and better rendering performance, making it suitable for user interfaces and interactive visualization systems [26].

In the project, labels for various robot components are displayed during object examination using TextMeshPro. To enhance user comprehension and interaction clarity, the matching label is shown in the 3D environment when a voice command is used to match a certain body part. TextMeshPro is great because it offers configurable real-time text rendering for interactive applications and interfaces directly with Unity.

III. SYSTEM DESIGN AND IMPLEMENTATION

3.1 System Architecture

The system uses a multimodal interactive architecture that integrates three-dimensional visualization, voice recognition, gesture recognition, and real-time communication into a unified inspection environment. The architecture is designed to combine many input modalities with real-time visual feedback to facilitate natural human-computer interaction. By distributing tasks across complementary interaction methods, multimodal architectures enhance usability, flexibility, and interaction efficiency, which is why they are frequently employed in interactive systems [26].

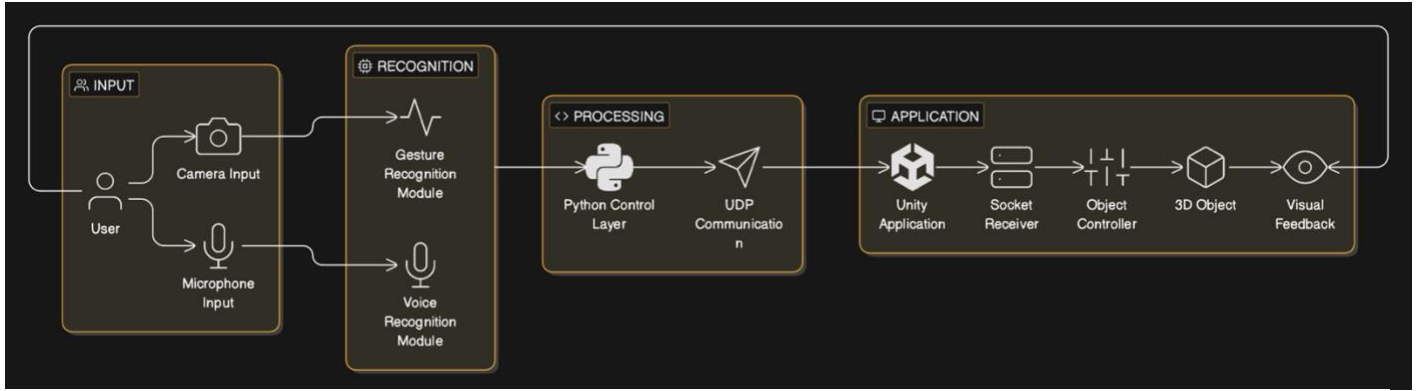


Figure 1 Presents the Architecture of the Developed Multimodal Interaction System.

Three primary subsystems make up the system's overall workflow: the input processing subsystem, the communication subsystem, and the visualization subsystem. The input processing subsystem is responsible for gesture recognition and voice recognition and is implemented in Python. While the SpeechRecognition library processes microphone input for speech command recognition, MediaPipe and OpenCV handle webcam input for hand tracking and gesture detection.

Once a gesture or voice command is detected, it is transmitted to the Unity environment through UDP socket communication. Real-time synchronization between the interaction modules and the visualization environment is made possible by the communication subsystem, which serves as a bridge between Python and Unity. UDP is a great fit because it provides low-latency transmission in real-time interactive systems.

The 3D robot is rendered, interaction commands are processed, and the projection-based visualization environment is managed by the visualization subsystem, which is implemented in Unity. After receiving commands, Unity translates them into object interactions like component-focused inspection, scene reset, rotation, and zooming. During interaction the system moreover controls item highlighting, label visualization, and camera movement.

The user and the projected visualization environment engage in an ongoing feedback loop through the interaction pipeline. When users use voice commands or gestures, the system interprets the input in real time, Unity modifies the visualization appropriately, and the revised scene is projected onto the real world. Within the developed 3D inspection system, this workflow supports intuitive and immersive interaction.

Because each subsystem functions independently while staying synced through the communication layer, the architecture's modular structure enhances maintainability and scalability. Future extensions like more gesture types, advanced speech processing, or augmented reality integration are also made possible by this approach without requiring major modifications to the overall system architecture.

3.2 Hardware and Software Environment

3.2.1 Hardware components

The system makes use of a hardware setup designed to facilitate projection-based visualization and real-time multimodal interaction. The setup consists of a webcam, microphone, projector, and a standard consumer-grade computer.

While the microphone records speech commands during engagement, the webcam captures hand gestures for real-time gesture detection. An interactive projection-based visualization space is created by projecting the Unity-generated 3D inspection environment onto a real wall surface using a digital projector. The Python processing subsystem, the Unity visualization environment, and real-time system component communication are all handled by the computer.

The selected hardware configuration provides an accessible and low-cost implementation while maintaining stable real-time interaction performance suitable for the proposed multimodal 3D object inspection system.

3.2.2 Software Components

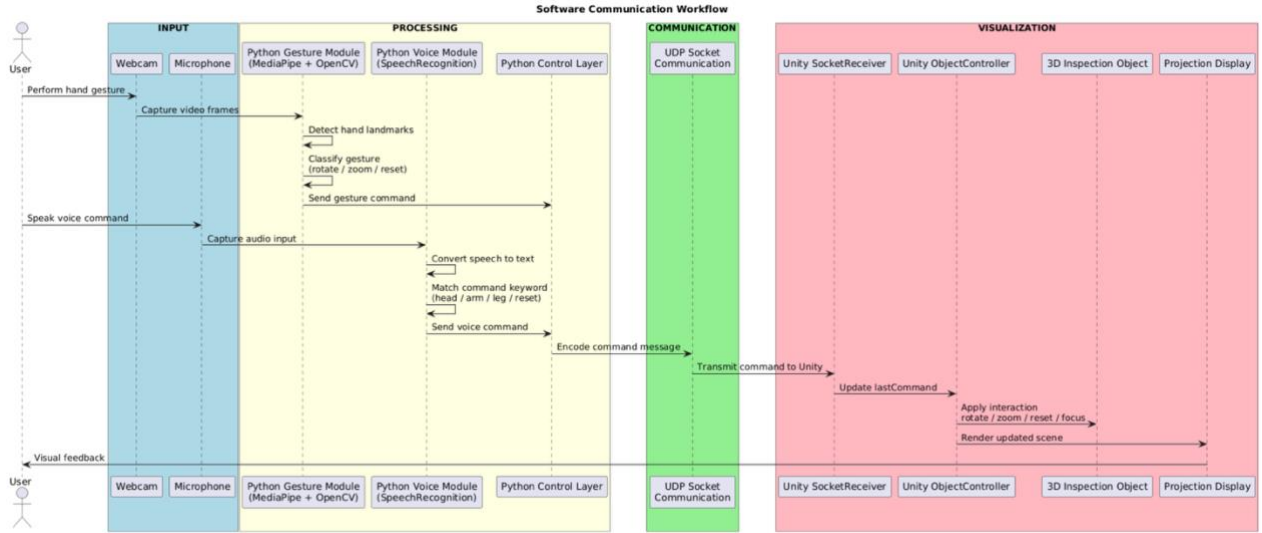


Figure 2 Illustration of the Software Communication Workflow

The system's software environment includes UDP socket communication, Unity, Python, MediaPipe, and SpeechRecognition. Together these elements enable projection-based visualization and real-time multimodal interaction.

The primary visualization tool for managing the projection display, handling object interaction, and rendering the 3D robot model is Unity. Python is used for implementing the gesture recognition and voice recognition subsystems. The SpeechRecognition library is utilized to interpret speech commands, and MediaPipe and OpenCV are integrated within Python to carry out real-time hand tracking and gesture detection.

Python and Unity can communicate via UDP socket communication, which enables real-time transmission and execution of recognized commands. The software environment's modular structure allows the recognition and visualization subsystems to remain synchronized while processing interaction tasks independently.

3.3 Unity Environment Development

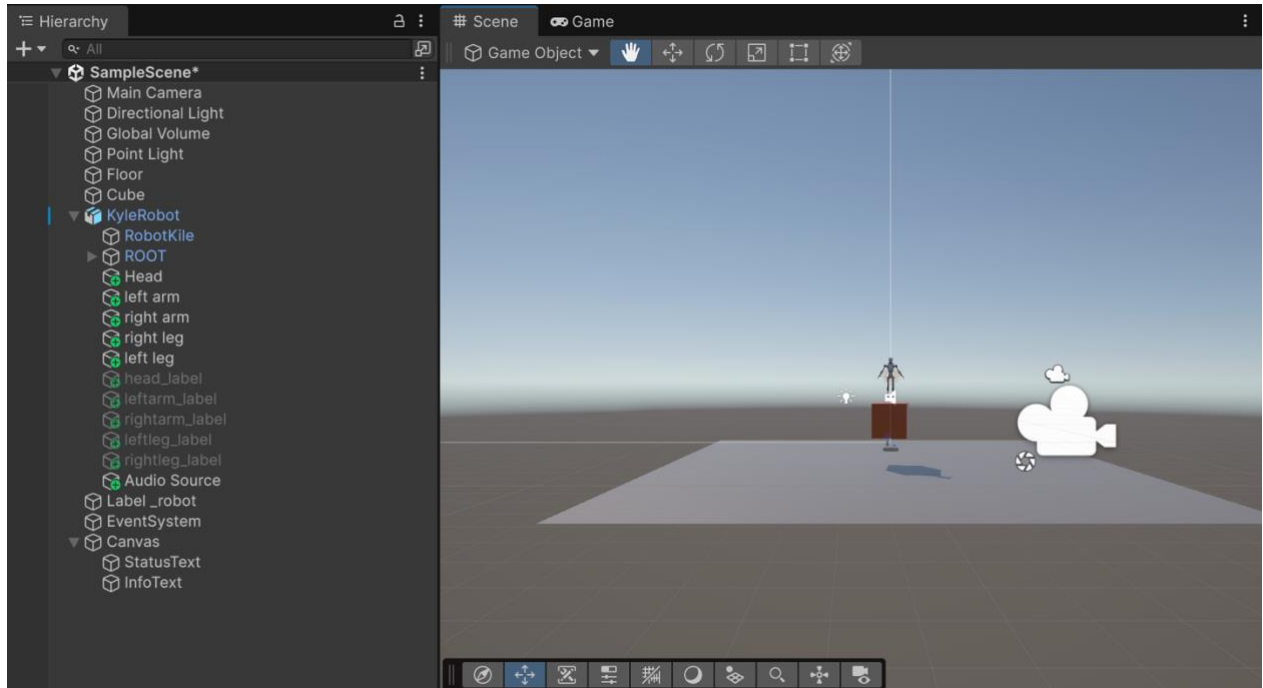


Figure 3 Hierarchy and Scene

The primary platform for the system's visualization and interaction is the Unity environment. It is responsible for processing interaction commands, rendering the three-dimensional robot model, and displaying the projection-based inspection environment in real time.

The robot model, lighting system, camera setup, and interaction components needed for object inspection are configured in the Unity scene. A directional lighting setup was used to improve object visibility and enhance the clarity of the projected visualization. During voice-controlled engagement, the Main Camera was set up to dynamically move toward specific robot components in order to facilitate object-focused inspection.

The main interactive object in the Unity scene was the robot model. C# was used to create interaction scripts that supported body-part inspection controls, rotation, zooming, and reset functionalities. Labels and highlighting mechanisms were also integrated to improve interaction feedback and user understanding during component-focused inspection.

The Python subsystem's real-time communication is managed by the Unity environment through UDP socket communication. Incoming gesture and voice commands are processed and translated into corresponding object interactions within the scene. The rendered output is then

projected onto a physical surface through the projector setup, creating the interactive visualization environment used in the system.

3.4 Gesture Recognition System

Real-time interaction between the user and the projected three-dimensional inspection environment is made possible by the gesture recognition system. Hand gestures are used as the primary interaction method for continuous object manipulation tasks including object rotation, zooming, and scene reset, hand gestures are the primary interaction method. Recent research highlights that real-time hand gesture recognition provides intuitive and natural interaction without requiring physical controllers, solidifying it as an important component of modern Human-Computer Interaction systems [30].

For real-time hand tracking and landmark detection, the implementation makes use of MediaPipe Hands. MediaPipe uses a standard RGB camera to estimate finger joint positions and hand orientation after detecting 21 hand landmark points from webcam input. Providing accurate real-time performance while maintaining low computational performance solidifies why modern MediaPipe -based tracking remains suitable for interactive applications.

3.4.1 MediaPipe Implementation

Using the MediaPipe hand tracking framework, the gesture recognition subsystem continually captures video frames from the webcam. The framework recognizes the user's hand and extracts landmark coordinates that correspond to the positions of the palm and finger joints. Then, gesture analysis and command generation are performed using these landmark points.

3.4.2 Hand Landmark Detection

The spatial positions of predefined hand keypoints serve as the basis for hand landmark detection. In real time, MediaPipe creates landmark coordinates for wrist locations, finger joints, and fingertips. The system can detect finger movement and identify gesture states using the collected landmark data without the need for specialized hardware like depth cameras or sensor gloves.

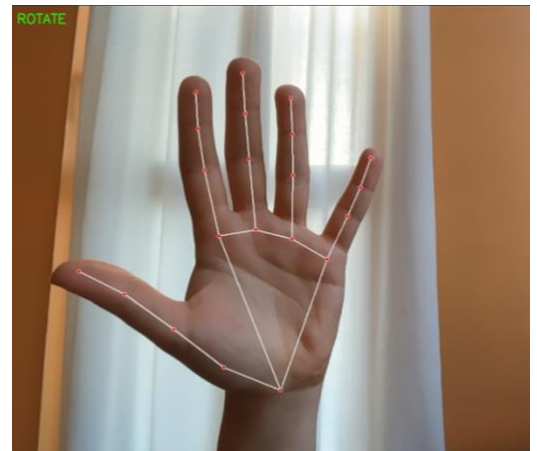


Figure 4 Illustration of Landmark Coordinates

Geometric correlations between landmark coordinates can be assessed in order to categorize particular gestures. The Euclidean distance between two landmark points is one measurement that is commonly used:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Where d is the distance between landmark points, and x_1, y_1 and x_2, y_2 are their coordinates.

In the implemented system, gesture states such as pinch gestures for zoom operations are identified by measuring the distance between selected fingertip landmarks. To distinguish between open-hand and close-hand motions, further landmark position analysis is utilized to determine whether fingers are folded or extended. Recent research indicates that landmark-based hand tracking provides accurate real-time interaction while maintaining low computational complexity, making it suitable for interactive HCI applications [33][34].

3.4.3 Gesture Detection

The system translates detected hand movements into interaction commands using a simple rule-based gesture classification approach. After landmark information is processed, predefined conditions are evaluated to determine the corresponding gesture and system action. This approach eliminates the need for computationally intensive deep learning classifiers while maintaining reliable real-time performance.

Pinch gestures are associated to zoom operations, close-hand gestures are used to trigger scene features, and open-hand gestures are mapped to object rotation inside the created system. Every gesture that is identified results in a corresponding command that is sent via UDP socket communication to the Unity environment.

In order to provide seamless interaction, gesture recognition systems built for HCI contexts often place a high priority on tracking stability and minimal latency. Research on real-time hand gesture recognition identifies computational efficiency and responsiveness as critical requirements for immersive interactive systems [30]. Consequently, the implementation supports continuous real-time object inspection, emphasizing fast gesture classification and efficient command generation.

3.4.4 Python Processing Workflow

Webcam image acquisition is the first step in the gesture recognition process. The MediaPipe framework constantly processes captured video frames to derive hand landmark coordinates in real time. The python processing subsystem then analyzes the landmark data and

evaluates predefined gesture conditions. When a valid gesture is detected, the corresponding command is generated and transmitted to the Unity environment through UDP socket communication. Real-time control of the projected 3D object is made possible by Unity, which receives the command and executes the corresponding interaction.

3.4.5 Rotation, Zoom, and Reset Gesture

Rotation, zoom, and reset are the three primary gesture-based interaction functions supported by the system. The zoom functionality enables a closer examination of object details, while rotation allows users to inspect the three-dimensional robot model from different viewing angles. The object and inspection environment are restored to their default state through the reset function.

Through communication between the Python processing subsystem and the Unity visualization environment, these interaction functions are mapped to predefined gesture commands and executed in real time. In the projection-based inspection system, this interaction structure provides responsive performance and intuitive object manipulation with low hardware requirements.

3.5 Voice Recognition System

In the developed multimodal inspection environment, the voice recognition system serves as a complementary interaction modality. Voice interaction is employed for discrete command execution and component-focused inspection actions, whilst gesture recognition is used for continuous object manipulation actions. Recent Human-Computer Interaction research emphasizes that speech-based interfaces improve accessibility and usability by enabling users to interact with digital systems through natural verbal commands without requiring physical input devices [35].

3.5.1 SpeechRecognition Integration

Real-time audio input from a microphone is captured and processed by the implementation using the Python SpeechRecognition module. The library uses voice-to-text recognition algorithms to translate recognized speech into text while continually listening for spoken commands. Because speech recognition frameworks enable hands-free engagement in real-time environments and offer lightweight implementation, they are frequently utilized in interactive systems [36].

3.5.2 Command Processing

The identified words are compared to a predefined command vocabulary following the conversion of speech input into text. The system maps particular keywords to associated interaction functions using a command-based recognition approach. This methodology improves recognition reliability by reducing ambiguity and limiting the number of possible command interpretations during runtime.

One way to illustrate the command mapping procedure is as follows:

$$C = \begin{cases} \text{HEAD} & \text{if } W = \text{"head"} \\ \text{LEFT_ARM} & \text{if } W = \text{"left arm"} \\ \text{RIGHT_ARM} & \text{if } W = \text{"right arm"} \\ \text{LEFT_LEG} & \text{if } W = \text{"left leg"} \\ \text{RIGHT_LEG} & \text{if } W = \text{"right leg"} \end{cases}$$

Where:

W represents the recognized speech input,

C represents the command generated by the system.

Once a valid keyword is detected, the corresponding command is generated and transmitted to the Unity environment for execution. Because reduced command structures increase processing speed, lower recognition errors, and enable more steady interaction performance in real-time environments, voice controlled HCI systems frequently adopt confined command vocabularies [35].

3.5.3 Body-Part Selection Commands

Voice commands are primarily used for component-focused object inspection. Predefined keywords such as “head”, “left arm”, “right arm”, “left arm”, and “right leg” are included in the implemented command set. When a command is recognized, the corresponding body part is selected within the Unity environment. After selection, the appropriate label and visual highlight are displayed as the camera automatically focuses on the designated component. Without requiring additional manual navigation, this interaction structure allows users to inspect individual robot components. In addition to reducing interaction complexity, separating voice commands from gesture-based manipulation adheres to multimodal interaction principles, which allocate continuous tasks to gesture interaction and symbolic tasks to speech input.

3.5.4 Real-time Communication with Unity

The Python processing subsystem generates a corresponding interaction command after a voice command is recognized and validated. UDP socket communication is used to send the command to the Unity visualization environment. Unity constantly monitors incoming commands and updates the inspection environment accordingly.

Real-time synchronization between the projected 3D visualization environment and the speech recognition subsystem is made possible by this communication workflow. Research on voice-interactive systems highlights that low-latency communication and fast command execution are essential for maintaining responsive user interaction and minimizing delays between user input and system response [36].

The developed multimodal projection-based environment integrates a voice recognition system to provide a lightweight and accessible solution for speech-based interaction while supporting efficient component-focused inspection.

3.6 Object Inspection and Interaction System

The real-time manipulation and examination of the projected three-dimensional robot model is made possible by the object inspection and interaction system. The system facilitates intuitive multimodal interaction by combining voice-controlled component selection with gesture-based object manipulation. Recent Human-Computer Interaction research enables direct manipulation of virtual objects within immersive environments; interactive object inspection systems enhance spatial understanding and user engagement [27].

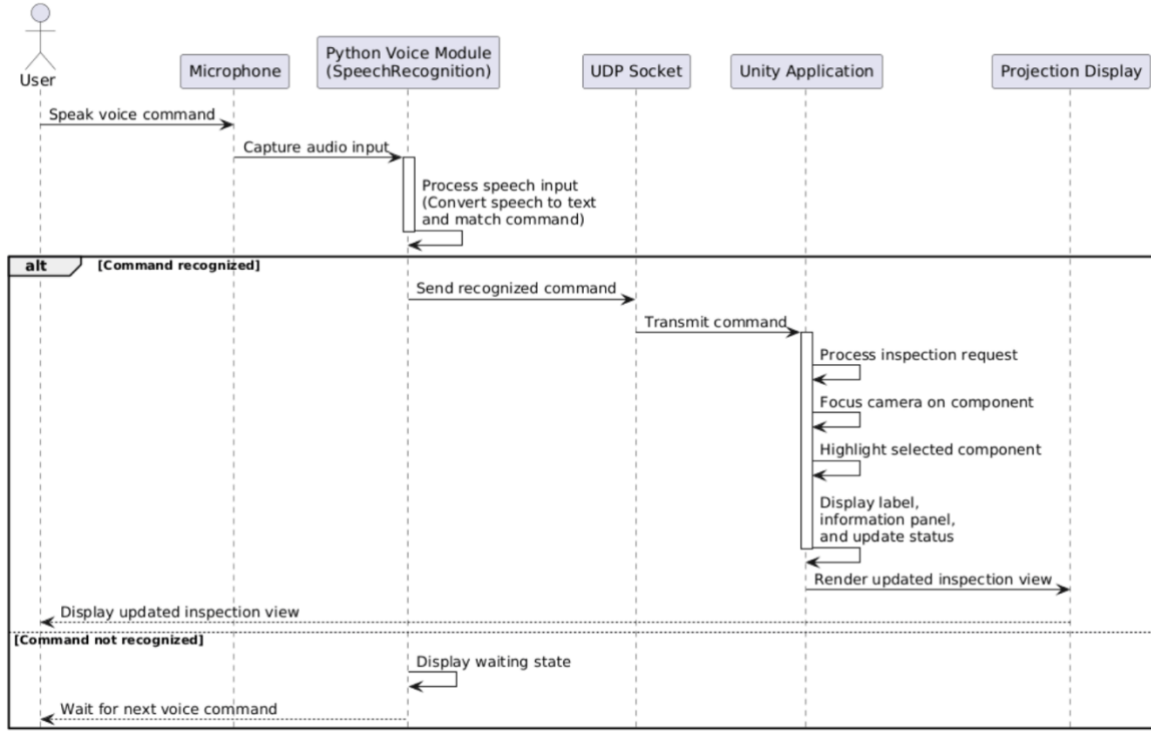


Figure 5 Voice Recognition and Component Inspection Process

3.6.1 Object Rotation System

Object rotation is implemented to allow users to inspect the robot model from different viewing angles. In the unity environment, rotation functionality is continuously applied to the object and is initiated through gesture-based interaction. Interactive rotation systems are commonly used in three-dimensional visualization environments because they improve spatial perception and enable more effective object exploration [27].

3.6.2 Zoom System

During inspection, users can alter the robot model's scale using the zoom mechanism. Closer examination of object details and improved visibility of individual components is accomplished by the zoom functionality. The object scale is dynamically updated in real time by the implemented zoom mechanism, which is triggered by gesture recognition. The functionality enhances inspection flexibility and allows users to focus on specific areas of interest within the visualization environment.

3.6.3 Camera focus system

The camera focus system is used during component focused inspection. The camera automatically repositions and orients itself in the direction of the selected body part when a voice command for body part selection is detected. By focusing the user's attention on relevant object regions, this method lessens the need for manual navigation and increases inspection efficiency. The camera orientation process can be represented using a viewing vector:

$$\vec{D} = \vec{T} - \vec{C}$$

Where:

$\vec{D}$ represents the viewing direction,

$\vec{T}$ represents the target object position,

$\vec{C}$ represents the camera position.

This calculation allows the camera to continuously face the selected inspection target within the Unity environment.

3.6.4 Label System

During inspection, the label system provides visual identification of selected robot components. When a body-part command is recognized, the corresponding label is displayed within the scene to indicate the currently selected component. During object inspection tasks, visual labeling improves user understanding and supports clearer interaction feedback. Research in interactive visualization environments highlights that contextual visual feedback improves usability and reduces interaction ambiguity during object exploration and analysis [37].

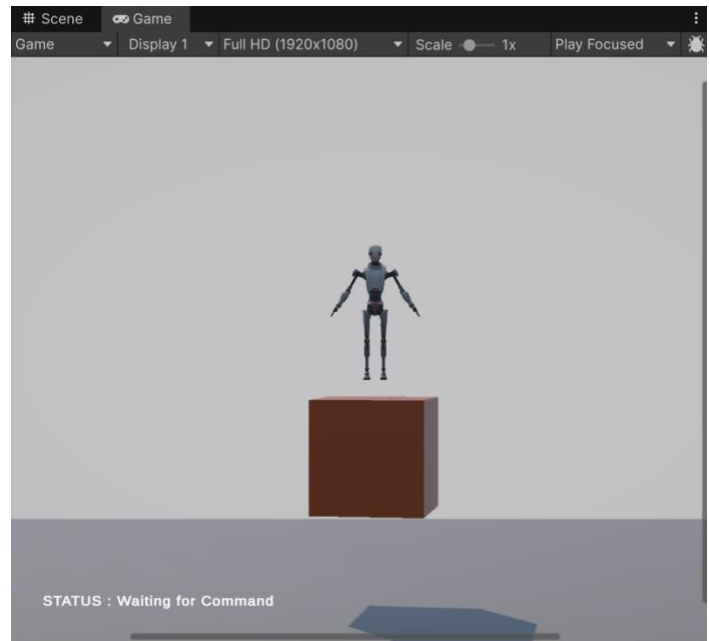


Figure 6 Initial Inspection Scene before Component Selection.

3.6.5 Highlight System

By adding a distinct material to the corresponding object region, the highlight system visually draws attention to the chosen robot component. Highlighting allows users to quickly identify the active inspection target and improves interaction clarity. Together with the label system, this feature enables instantaneous visual feedback following voice-command selection.

3.6.6 Component-focused Inspection

Voice interaction, camera focusing, labeling, and highlighting are all integrated into a unified inspection workflow in component-focused inspection. After a user selects a robot component through a voice command, the corresponding label and highlight are activated as the camera automatically focuses on the selected region. Without manually repositioning the camera or navigating the scene, users can examine individual robot components because of this interaction framework.

Combining multiple feedback modalities enhances usability, interaction efficiency, and user understanding in immersive visualization systems, according to recent multimodal interaction research [27].

The implemented object inspection and interaction system maintains responsive multimodal interaction and low hardware requirements while enabling intuitive real-time exploration of the projected three-dimensional robot model.

3.7 Projection-Based Visualization

The interactive three-dimensional inspection environment is displayed in a physical space through the projection-based visualization subsystem. The Unity-generated scene is projected onto a wall surface using a digital projector to create a shared visualization environment where users can interact with the projected robot model via voice commands and gestures. Projection-based visualization systems link digital content with physical environments without requiring wearable display devices, they are commonly utilized in HCI applications [4].

3.7.1 Projector Integration

The developed system's primary output device is the projector. The rendered scene generated by the Unity environment is transmitted directly to the projector and displayed on a wall surface. This configuration preserves a natural viewing experience while allowing users to

view interaction results in real time. Unlike head-mounted displays, projection-based visualization supports collaborative viewing and allows multiple users to observe the inspection process simultaneously.

3.7.2 Visualization Alignment

The projected image was aligned with the available wall surface using simple calibration techniques to ensure accurate display of the projected content. Alignment involved adjusting projector position, projection distance, image scale, and projection angle to achieve clear visibility of the projected robot model. In projection-based systems, proper alignment is crucial since it lowers visual distortion during interaction and enhances visualization quality.

3.7.3 Projection Setup

The projection setup consisted of a computer running the Unity application, a digital projector, and a surface used as the projection area. The projector was positioned to ensure an adequate image size for object inspection while offering a good view of the projected environment. The projection area was chosen to minimize environmental interference and ensure sufficient visibility under normal room lighting conditions.

3.7.4 Interactive Display Environment

The user's main area of engagement is the projected visualization environment. All object manipulations, camera movements, highlighting operations, and component inspections are displayed directly within the projected scene. Through this method, digital content is still accessible while fostering a shared engagement environment.

Instead of depending on immersive wearable technology, research on projection-based interaction settings shows that projection systems can enhance accessibility and user comfort by directly integrating digital information into the environment [4].

3.7.5 Real-Time Projected Interaction

Real-time interaction is achieved through continuous communication between the gesture recognition subsystem, voice recognition subsystem, and Unity visualization environment. A corresponding interaction is processed and immediately reflected within the projected scene, when a user performs a gesture or issues a voice command. With immediate visual feedback, this approach allows users to rotate, zoom, reset, and inspect individual robot components. The combination of projection-based visualization and multimodal interaction supports natural user

interaction and enhances the overall usability of the developed system while providing an intuitive inspection.

IV. USER MANUAL AND ANALYSES OF EXPERIMENTAL RESULTS

4.1 User Manual

The User Manual provides guidance for operating the developed multimodal projection mapping-based 3D object inspection system. It describes the hardware and software requirements, system startup procedure, gesture and voice interactions, and the projection-based inspection workflow. The purpose of this section is to assist users in effectively utilizing the system for real-time object inspection.

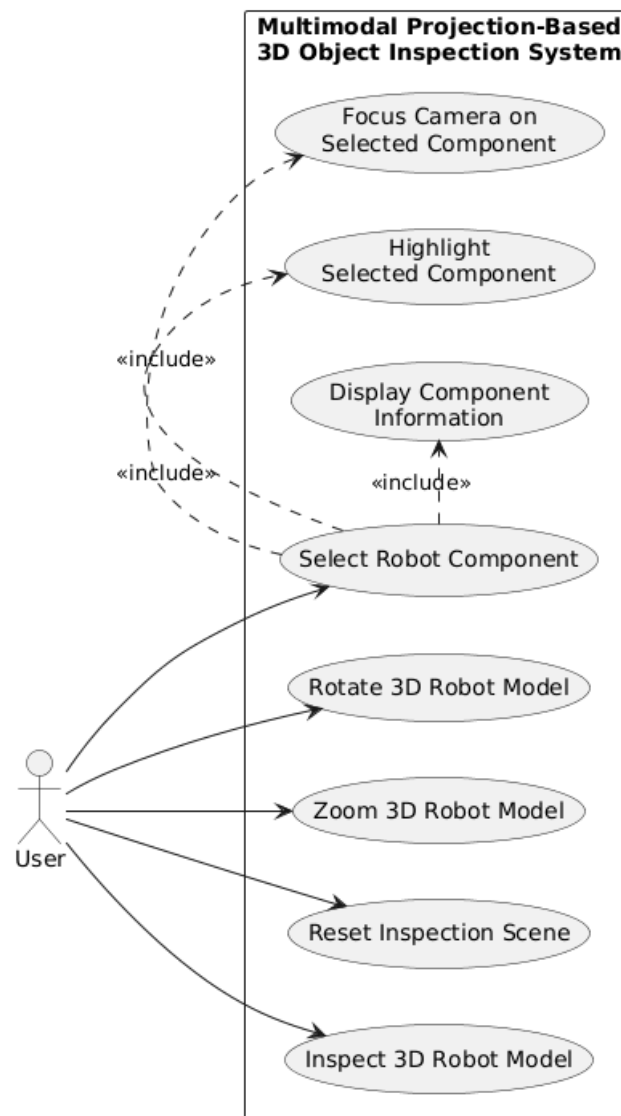


Figure 7 Use Case Diagram

4.1.1 System requirements

To enable gesture detection, voice recognition, real-time visualization, and projection-based interaction, the developed multimodal projection mapping-based 3D object inspection system requires a combination of hardware and software components. The minimum hardware and software requirements for operating the developed system are presented below.

Hardware Requirements

The hardware configuration consists of a standard computer, webcam, microphone, and projector. The microphone is used to receive voice commands, while the webcam is responsible for capturing hand movements for gesture recognition. Users can engage with the system in a projected environment through the projector, which projects the interactive 3D robot model onto a projection surface.

The hardware components used on the development system are summarized in Table 4.1. they were selected to provide a low-cost and accessible platform for multimodal interaction

COMPONENT	PURPOSE
Laptop	Executes the Unity and Python Subsystems
Webcam	Captures hand gestures for real-time tracking
Microphone	Receives user voice commands
Projector	Displays the interactive 3D visualization
Projection Surface	Provides the display area for projection mapping

Table 4.1 Hardware Components Used

Software Requirements

The software environment consists of Unity for 3D visualization and Python for gesture and voice processing. UDP socket communication is used to establish communication between the two environments, enabling the real-time exchange of interaction commands.

The software components required to operate the system are presented in Table 4.2.

SOFTWARE COMPONENT	PURPOSE
Unity	Development and visualization environment
Python	Gesture and voice processing subsystem
MediaPipe	Hand landmark detection and tracking
SpeechRecognition	Voice command recognition
UDP Communication Module	Real-time data exchange between Python and Unity
Visual Studio Code	Script development and debugging

Table 4.2 Software Components for the System

The system was designed to function with widely accessible hardware and software resources, eliminating the need for specialized wearable devices, depth cameras, or dedicated tracking equipment. While preserving responsive multimodal interaction and real-time object inspection capabilities, this configuration reduces implementation costs.

4.1.2 System Startup Procedure

All hardware devices must be properly connected and configured before interacting with the developed inspection system. For the gesture recognition, voice recognition, and projection-based visualization modules to function properly, the webcam, microphone, projector, and computer are necessary.

The system startup process begins by connecting the webcam and microphone to the computer and positioning the projector to display the virtual robot model on the designated projection surface. The Python subsystem responsible for gesture and voice processing is started after the hardware configuration is finished. After launching the subsystem initializes the gesture and voice recognition modules and prepares the system for user interaction.

The Unity application is launched once the Python subsystem has been successfully initialized. The 3D robot model, user interface (UI) elements, camera control system, and projection visualization environment are all loaded by Unity. The object inspection module, which includes component labels, highlighting features, status indicators, and the information panel, is also initialized by the application during startup.

UDP socket communication is used to establish communication between the Python and Unity subsystems. The Python application continuously monitors gesture and voice inputs and instantly transmits recognized commands to Unity in real time. Unity executes the appropriate

interaction, such as object rotation, zooming, scene resetting, or component-focused inspection, when it receives a command.

The robot model appears in the projection environment, and the status indicator displays a waiting status after initialization is successfully finished. The system then enters operational mode and becomes ready to receive gesture and voice commands from the user.

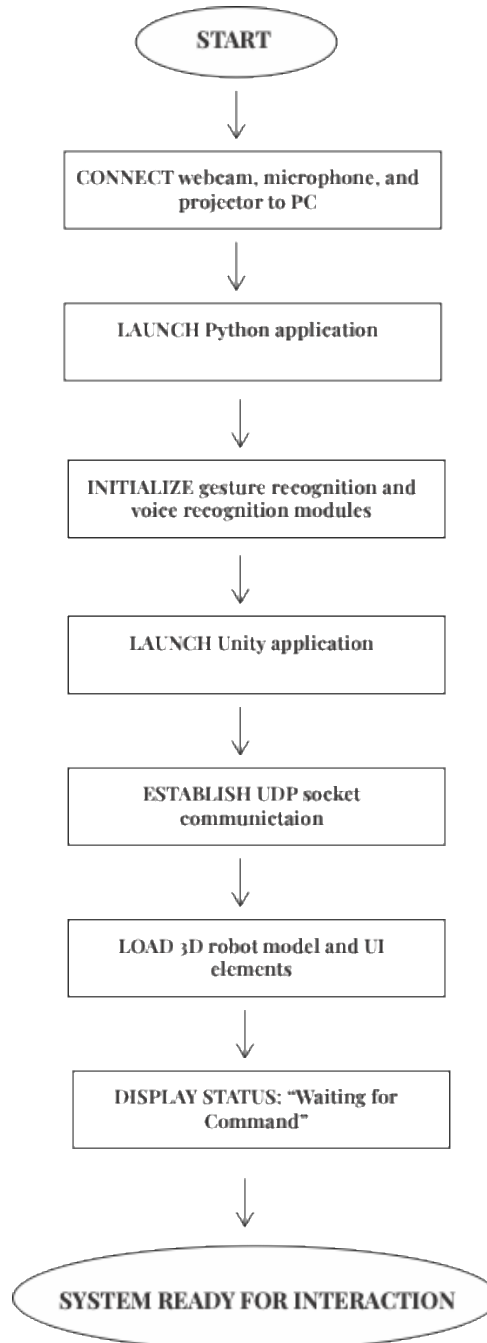


Figure 8 System Startup Procedure of the Developed System

4.1.3 Gesture-Based Interaction

In order to facilitate intuitive manipulation of the 3D robot model without the need for conventional input devices, the created system offers gesture-based interaction. The MediaPipe hand-tracking framework processes user hand movements that are captured using a webcam. the user performs gestures in front of the webcam. Once recognized, the corresponding interaction is executed automatically within the Unity environment. The gesture interaction module allows users to manipulate the robot model directly within the projection environment. The system uses three main gestures: scene resetting, object rotation, and object zooming.

Rotation Gesture

To rotate the robot model, the user performs an open-hand gesture in front of the webcam. The robot model is continuously rotated around its vertical axis by the algorithm upon detecting an open-hand motion. Without having to manually move the camera, this functionality enables users to examine the object's structure and view various parts.

The status indicator provides visible confirmation that the rotation command has been successfully acknowledged during operation by displaying the active interaction state.

Zoom Gesture

To zoom toward the robot model, the user performs a pinch gesture. The pinch movement that initiates the gesture is identified by measuring the distance between specific fingertip landmarks. The camera gradually moves toward the robot model upon recognition of the gesture, enabling users to examine object details in greater detail.

The zoom functionality improves inspection accuracy by providing a clearer view of individual robot components and enhancing the visibility of selected regions.

Reset Gesture

To restore the default inspection state, the user performs a close-hand gesture. When a close-handed gesture is detected, the system resets the camera position, camera orientation, robot rotation, and object scale to their default values. Additionally, any active labels, highlights, or component selections are removed.

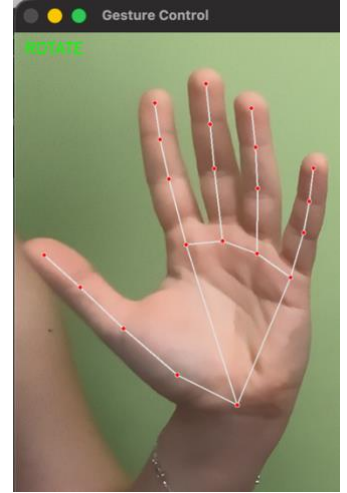


Figure 9 Rotation Gesture



Figure 10 Zoom Gesture

Without having to manually reposition the object or camera, this feature enables users to swiftly return to normal inspection view.

Within the projection-based visualization environment, the developed gesture set offers a straightforward and user-friendly interaction mechanism that facilitates effective object exploration while preserving real-time responsiveness.

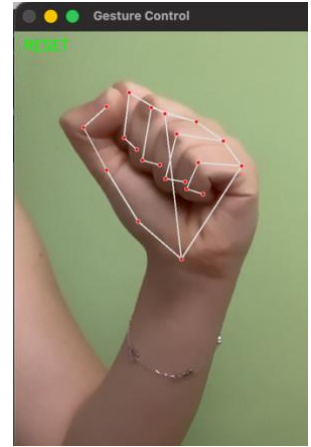


Figure 11 Reset Gesture

4.1.4 Voice-Based Interaction

Voice-based interaction is supported by the developed system, allowing users to select specific robot components without the need for traditional input devices. The microphone captures voice commands, which the system then automatically processes. When a legitimate command is identified, the associated action is carried out in the Unity environment, enabling users to effectively examine various aspects of the robot model.

Voice interaction complements gesture-based manipulation by providing a convenient method for navigating directly to specific robot components. Instead of manually repositioning the camera, users simply pronounce the desired component name, and the system automatically performs the inspection operation.

The supported voice commands are summarized in Table 4.4.

VOICE COMMAND	SYSTEM ACTION
Head	Focuses the camera on the robot head, highlights the selected component, and displays the corresponding information.
Left Arm	Focuses the camera on the left arm, highlights the selected component, and displays the corresponding information.
Right Arm	Focuses the camera on the right arm, highlights the selected component, and displays the corresponding information.
Left Leg	Focuses the camera on the left leg, highlights the selected component, and displays the corresponding information.
Right Leg	Focuses the camera on the right leg, highlights the selected component, and displays the corresponding information.

Table 4.4 Supported Voice Commands

Component Selection

To inspect a specific robot component, the user speaks one of the supported voice commands while the system is in the ready state. The camera automatically transitions toward the selected component after the command is recognized, providing a closer inspection view. Simultaneously, the selected component is visually highlighted, and its corresponding label and information panel are displayed within the user interface.

These visual feedback systems enhance the clarity of the inspection process while enabling users to quickly verify that the command has been successfully recognized. If another component is selected, the system automatically updates the camera position, highlighting effect, and displayed information to reflect the newly selected component.

The voice interaction module makes it possible to navigate the robot model efficiently and eliminates the need for manual camera adjustments, making the inspection process more intuitive and user-friendly.

4.1.5 Projection-Based Inspection

The developed system offers an interactive environment for inspecting three-dimensional objects using projection-based visualization. The robot model is projected onto a wall, to demonstrate the technology enables users to interact with the visualization naturally through gesture and voice commands, instead of requiring wearable devices such as augmented reality headsets or specialized tracking equipment. This approach preserves real-time responsiveness while creating an intuitive inspection environment.

Projector Setup

Before starting the inspection process, the projector should be positioned to ensure that the projected robot model is clearly visible on a flat projection surface. Excessive ambient lighting and physical obstructions that may reduce image quality or interfere with user interaction should be removed from the projection area.

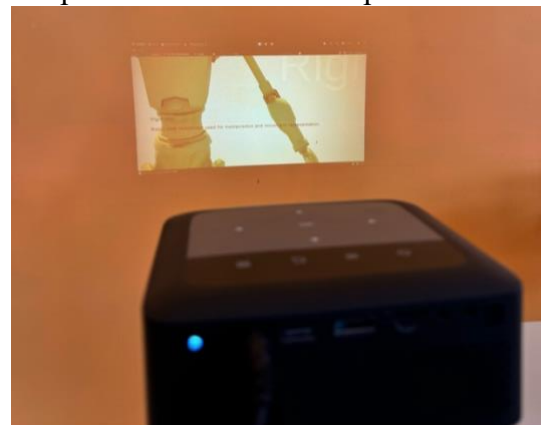


Figure 12 Projection-Based Visualization Setup

After the projector is configured, the Unity application displays the 3D robot model together with the user interface elements, including the status indicator, component labels, and information

panel. The system enters the ready state and waits for user commands after the projection is properly aligned.

Inspection Workflow

The projection-based inspection process consists of a sequence of user interactions that enable efficient examination of the robot model. The typical workflow is summarized below:

1. Start the system and verify that the projected robot model is displayed correctly.
2. Use gesture commands to rotate or zoom the robot model for general inspection.
3. Issue a supported voice command to select a specific robot component.
4. The system automatically focuses the camera on the selected component.
5. The selected component is highlighted, and its corresponding label and information panel are displayed.
6. Continue inspecting additional components or manipulate the model using gesture commands.
7. Use the reset gesture to restore the default inspection view when necessary.

Without using conventional input devices such as a mouse or keyboard, this workflow allows users to transition between global object manipulation and detailed component inspection with ease.

The projection-based inspection process combines gesture interaction, voice interaction, camera focusing, component highlighting, and information display into unified multimodal inspection environment. These characteristics improve the system's overall usability by allowing users to immediately identify the selected component and comprehend the current interaction state through visual feedback.

4.2 Experimental Setup

The experimental setup was designed to evaluate the functionality and real-time performance of the developed multimodal projection mapping-based 3D object inspection system. Verifying the integration of gesture recognition, voice recognition, projection-based visualization, and object inspection within a unified interaction environment was the focus of the experimentation tests. Following the integration and debugging procedure outlined in Chapter 3, all tests were carried out using the system's final implementation.

4.2.1 Testing Environment

The multimodal interaction was made possible by the hardware configuration, software configuration, and projection-based visualization environment that made up the testing environment. The objective of the testing environment was to simulate the intended operating conditions of the developed system while evaluating its ability to respond to user gestures and voice commands in real time.

Hardware Configuration

A laptop computer, a webcam, a microphone, and a digital projector are part of the hardware configuration. The laptop executed both the Unity application and the Python subsystem simultaneously. While the microphone received voice commands for robot component selection, the webcam continuously captured the user's hand movements for gesture recognition. In order to enable interaction inside a projected environment without the need for wearable technology or specialized tracking hardware, the projector projected the 3D robot model onto a projection surface.

Software Configuration

The software configuration consisted of two cooperating subsystems. The first subsystem, implemented in Python, was responsible for processing gesture and voice inputs using the MediaPipe and SpeechRecognition libraries. The visualization environment, camera control, object manipulation, highlighting, status indicator, and component information panel were all managed by the second subsystem, which was built in Unity.

UDP socket communication was used to establish communication between the two subsystems. During testing, the Python application continuously monitored user input and transmitted recognized commands to Unity, where the corresponding interaction was executed in real time. A low latency communication between input processing and visualization was established by this architecture.

Projection Environment

The robot model was projected onto a wall (a flat projection surface) using the projector in an indoor environment. The webcam and microphone were positioned in front of the user to enable comfortable gesture and voice interaction throughout the testing process. The projection area was set up to ensure that the projected model could be clearly visible while reducing environmental interference.

During testing, users interacted with the projected robot model using predefined gestures and voice commands. Gesture interaction enabled object rotation, camera zoom, and scene reset, while voice interaction allowed direct selection of robot components. When a component was selected, the information panel was activated, the camera was automatically focused on the corresponding region, visual highlighting was applied, and the component label was displayed. Throughout the interaction process, the status indicator provided the continuous feedback regarding the current system state.

4.2.2 Test Scenarios and Experimental Evaluation

A series of experimental test scenarios was conducted, to evaluate the functionality and reliability of the developed multimodal projection mapping-based 3D object inspection system. Each scenario was designed to evaluate a particular subsystem and verify how gesture recognition, voice recognition, and projection-based visualization are integrated. All tests were performed using the final implementation after the completion of system integration and refinement.

Before the final evaluation, several issues were identified during development phase through iterative testing. Early tests showed that the camera zoom function did not consistently focus on the center of the robot model, occasionally directing the view toward the lower portion of the object. To improve the inspection experience, the camera movement logic was modified so that zoom operations dynamically targeted the center of the robot model.

During development, the reset functionality also needed to be refined. Early testing indicated that executing a reset command restored some object properties but did not always return the camera to its original inspection position and orientation. In order to guarantee a consistent starting point for subsequent interactions, the reset mechanism was enhanced to return the camera position, camera orientation, robot rotation, and object scale to their default state.

During component-focused inspection, additional adjustments were necessary to improve camera alignment and visual feedback. Initial testing revealed that the information panel was visible even when no robot component was actively selected, and the camera did not always focus precisely on the selected robot component. After that, the inspection workflow was refined so that when a component is selected, the camera automatically focuses on the desired region, visual highlighting is applied, the associated label and information panel are displayed, and the status

indicator is updated. Likewise, the information panel was set up to only show up while selecting components and to disappear when the system goes back to its initial configuration.

After these improvements, the completed system was put through four experimental test scenarios designed to evaluate its major functionalities.

Scenario 1: Gesture Recognition Testing

The gesture recognition subsystem was evaluated in the first scenario. The user performed predefined hand gestures in front of the webcam while the system analyzed the detected hand landmarks and generated the corresponding interaction commands. The following interactions were tested:

- Object rotation using the open-hand gesture.
- Camera zoom using the pinch gesture.
- Scene reset using the close-hand gesture.

To assess identification consistency and system responsiveness, each gesture was performed multiple times under normal operating conditions. During testing, it was observed that environmental factors such as insufficient lighting, rapid hand movement, use of two hands, partial hand occlusion, or improper camera positioning could occasionally affect recognition accuracy. Reliable detection was typically achieved

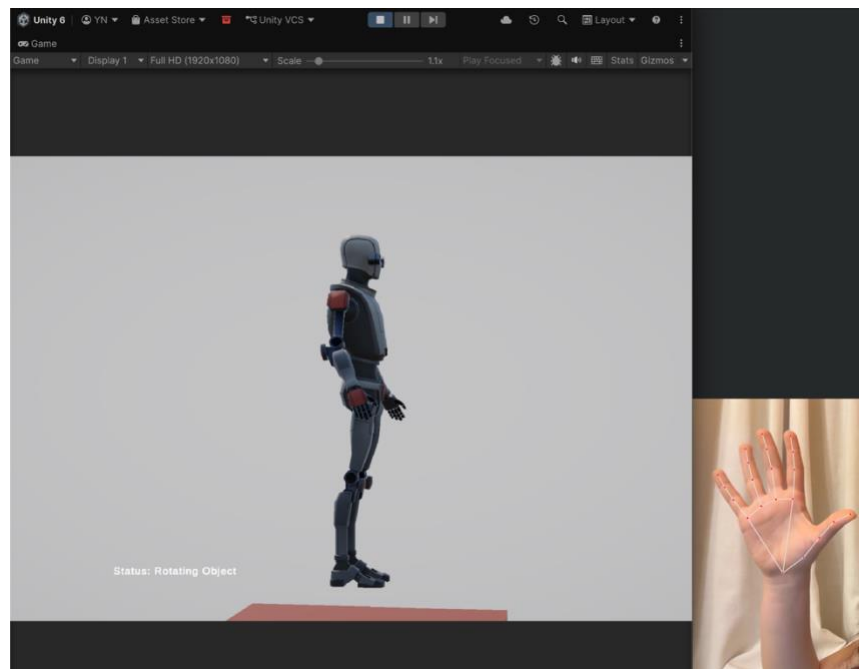


Figure 13 Gesture-Based Rotation of the 3D Robot Model

by keeping the hand in a clear position within the webcam's field of view.

Overall, the gesture recognition subsystem demonstrated stable performance under normal indoor conditions. Minor delays were observed only under unfavorable environmental conditions such as poor lighting or improper hand positioning.

GESTURE	TRIALS	SUCCESSFUL	OBSERVATION
Rotation	10	10	Smooth and responsive
Zoom	10	9	One delayed recognition under poor lighting
Reset	10	10	Successfully restored initial state

Table 4.5 Gesture Recognition Results in a Proper Environment with no Disturbances

Scenario 2: Voice Recognition Testing

The second scenario evaluated the voice recognition subsystem. The user used the supported voice commands, while the microphone captured the spoken input for processing. The following commands were tested:

- Head
- Left Arm
- Right Arm
- Left leg
- Right leg

The system's capacity to accurately identify the spoken command and initiate the associated component-focused inspection was the criteria for successful

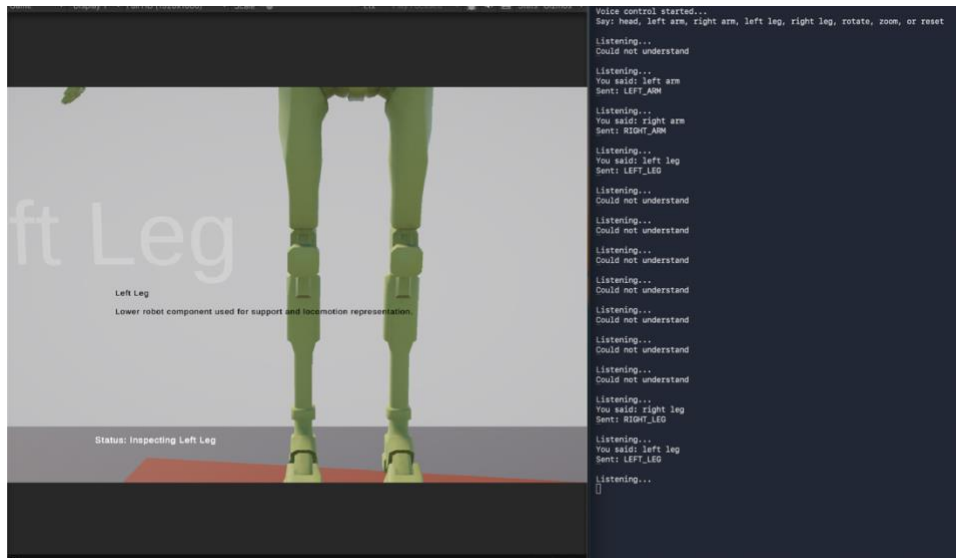


Figure 14 Voice Command-Based Component Inspection of the Left Leg

recognition. Testing demonstrated reliable performance in quiet environments, although when background noise was present or commands were delivered too fast or unclearly, there were occasional delays or incorrect recognition. In most cases, these issues were resolved by repeating the command or improving pronunciation.

Overall, the voice recognition subsystem successfully identified the supported commands in most trials. Recognition performance was affected primarily by pronunciation variations and environmental noise, which occasionally required the user to repeat the command.

VOICE	TRIALS	SUCCESSFUL	OBSERVATION
Head	10	6	Problem with pronunciation
Left Arm	10	9	Did not recognize command
Right Arm	10	8	Problem with pronunciation
Left Leg	10	9	Minor delay due to pronunciation
Right Leg	10	9	Did not recognize command

Table 4.6 Voice Recognition Results in a Quiet Environment

Scenario 3: Object Inspection Testing

The integrated object inspection functionality was evaluated in the third scenario. The system was expected to execute several coordinated operations simultaneously, once a valid voice command was recognized. The following functions were verified:

- Automatic camera focusing on the selected component.
- Visual highlighting of the selected component.
- Display of the corresponding component label.
- Display of the component information panel.
- Update of the status indicator reflecting the current interaction.

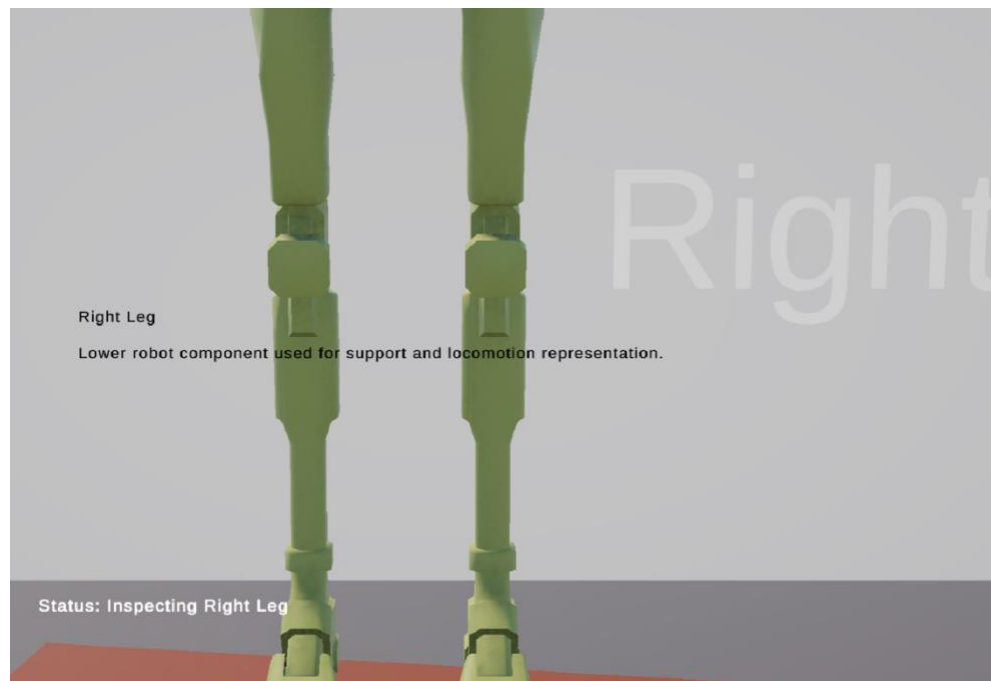


Figure 15 Component Highlighting, Label Display, and Information panel during Right Leg Inspection

Overall, the integrated object inspection subsystem operated successfully after the implemented refinements. The system consistently focused on the selected component while simultaneously applying visual highlighting, displaying the corresponding label and information panel, and updating the status indicator. These coordinated operations provided clear visual feedback and improved the overall inspection experience.

FUNCTION EVALUATED	RESULT	OBSERVATION
Camera Focusing	Successful	Accurately focused on selected component
Highlight	Successful	Visible highlight
Component Label Display	Successful	Displayed correct label
Information Panel	Successful	Displayed only during component selection
Status Indicator	Successful	Correctly reflected current interaction state

Table 4.7 Object Inspection Results

Scenario 4: Projection-Based Visualization Testing

The final scenario evaluated the projection-based visualization environment while gesture and voice interactions were performed. The objective was to determine whether the projected visualization remained stable, responsive, and suitable for real-time object inspection. The following aspects were evaluated:

- Visibility of the projected robot model.
- Alignment of the projected visualization.
- Responsiveness of real-time interactions.
- Clarity of component labels, highlighting effects, and the information panel.
- Overall usability of the projection-based inspection environment.

Throughout testing, multimodal interaction was effectively facilitated by the projection-based visualization. The projected visualization remained stable during system operation, while the component labels, highlighting effects, information panel, and status indicator provided clear visual feedback throughout the inspection process.

However, the quality of the projected visualization was impacted by several environmental conditions. Ambient lighting influenced image visibility, with excessive room brightness reducing

projection contrast. Additionally, in order to preserve proper visualization, a slight projector misalignment occasionally required manual adjustment. Since an older projector was used during experimentation, a minor delay in updating the projected image was also observed during some interactions. This delay affected only the visualization speed and did not interfere with the accuracy of gesture recognition, voice recognition, or object inspection.

Overall, real-time 3D object inspection was successfully handled by the projection-based visualization subsystem. Despite minor hardware and environmental limitations, the developed system provided a stable, intuitive, and user-friendly multimodal interaction experience.

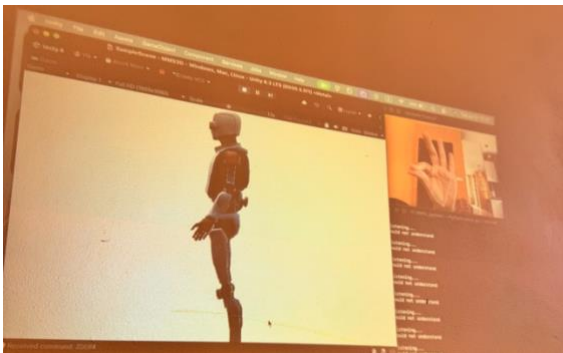


Figure 16 Projection-Based Gesture Interaction

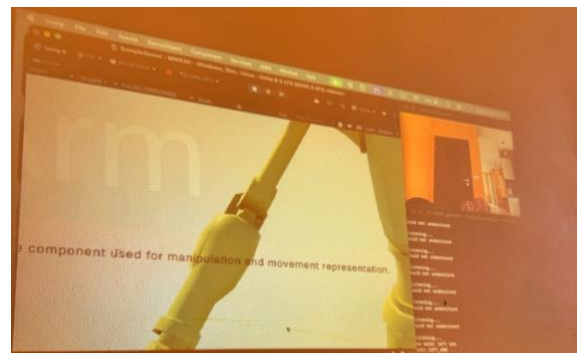


Figure 17 Projection-Based Voice Interaction

ASPECT EVALUATED	RESULT	OBSERVATION
Projected Model Visibility	Successful	Clearly visible under indoor lighting conditions
Projection Alignment	Successful	Minor manual adjustment occasionally required before testing
Real-Time Interaction	Satisfactory	Small delay observed due to the projector's refresh rate, but interactions remained accurate
Labels and Highlighting	Successful	Clearly displayed and synchronized with component selection
Information Panel	Successful	Displayed only during component focused inspection and hidden otherwise
Overall Usability	Good	The system operated reliably despite the projector's hardware limitations

Table 4.8 Projection-Based Visualization Results

MODULE	OBJECTIVE	ACHIEVED	REMARKS
Gesture Recognition	Object manipulation	yes	Sensitive to lighting and hand positioning
Voice Recognition	Component selection	yes	Performance depends on pronunciation and ambient noise
Camera Focus	Automatic inspection	yes	Refined to correctly center the selected component
Highlighting System	Visual emphasis	yes	Correctly identifies selected component
Label System	Component identification	yes	Labels displayed correctly
Information Panel	Component information	yes	Visible only during component selection
Status Indicator	User feedback	yes	Reflects current interaction state
Projection Visualization	Real-time display	yes	Slight delay due to older projector hardware
Integrated Multimodal System	Real-time 3D object inspection	yes	Successfully combines gesture recognition, voice recognition, and projection-based visualization

Table 4.9 Overall Experimental Performance of the Developed System

4.3 Discussion of Results

The created multimodal projection mapping-based 3D object inspection system effectively achieved the objectives established at the beginning of the study, as the experimental evaluation demonstrated. The system allowed users to inspect and manipulate a 3D robot model using natural interaction methods rather than conventional input devices by combining gesture detection, speech recognition, and projection-based visualization into a unified environment.

One of the major strengths of the developed system is the complementary use of multiple interaction modalities. While speech recognition was assigned to discrete component selection, gesture recognition was used for continuous manipulation tasks. This distribution of responsibilities simplified the interaction process and allowed users to perform inspection tasks in

a more intuitive manner. By offering instantaneous visual feedback, the automatic camera focus, component highlighting, label visualization, status indicator, and information panel significantly improved the inspection process's clarity.

Another significant contribution of the system is its accessibility. The created implementation uses a conventional computer, webcam, microphone, and projector, in contrast to many existing multimodal interaction systems that rely on wearable hardware, augmented reality headsets, or specialized tracking devices. This demonstrates the real-time visualization and object inspection is possible while achieving effective multimodal interaction using low-cost and widely available hardware.

The evaluation also identified several practical limitations that should be considered. The system performance remains dependent on environmental conditions such as lighting, background noise, camera positioning, and projector alignment. Furthermore, the features of the projection hardware affect the quality of the visualization, and using an older projector caused slight delays in the projected display. However, rather than the correctness of the features that were executed, these characteristics had an impact on the user experience.

Overall, the results indicate that the developed system provides a reliable and user-friendly environment for multimodal 3D object inspection. The successful integration of gesture interaction, voice commands, and projection-based visualization demonstrates the feasibility of combining multiple natural interaction techniques within a single low-cost platform. Consequently, the proposed implementation represents a practical contribution to multimodal HCI and establishes a foundation for future enhancements and more advanced projection-based inspection systems.

CONCLUSION

This study presented the design and development of a multimodal projection mapping-based 3D object inspection system that integrates gesture recognition, voice recognition, and projection-based visualization into a unified Human-Computer Interaction environment. The primary objective of the research was to create an easy-to-use inspection tool that would enable users to analyze and manipulate a three-dimensional robot model using natural interaction methods rather than traditional devices. This objective was successfully achieved by using a UDP socket technology to communicate between a Unity-based visualization environment and Python-based gesture and voice processing.

The developed system allows users to execute component-focused inspection by employing voice commands and predetermined voice commands and predefined hand gestures to rotate, zoom, and reset the 3D object. The system instantly focuses the camera on the selected component, applies visual highlighting, displays the corresponding label and information panel, and updates the status indicator to provide the user instant feedback when it detects a valid command. The system provides an interactive and user-friendly method to examine 3D objects without the need for wearable devices or specialized tracking equipment by incorporating these features into a projection-based environment.

The system successfully carried out its intended functions and successfully integrated gesture detection, voice recognition, and projection-based visualization into a unified multimodal interaction platform, according to the experimental evaluation. Although environmental conditions such as lighting, background noise, and projector characteristics influenced the overall user experience, the implemented refinements resulted in stable and responsive system performance under normal operating conditions. Additionally, several implementation issues were found and fixed contributing to the iterative development approach, which improved the final system's reliability and usability.

Beyond the implemented prototype, the proposed system has significant potential for application in various fields that require interactive 3D visualization and inspection. The device could assist practitioners examine three-dimensional reconstructions of patient scans, allowing natural gesture and voice interaction examine anatomical structures. In architecture and real estate, it could serve as an interactive presentation tool that enables clients to inspect building models, apartment layouts, or construction designs in a detailed 3D environment rather than relying solely

on static images or 2D plans. Similarly, the suggested approach can also be applied to engineering, robotics, education, industrial training, and product design, whether object inspection and intuitive visualization can enhance analysis, learning, and decision-making.

The study further demonstrated that multimodal interaction systems can be implemented using affordable and widely available hardware while still providing effective and engaging user experience. Compared to using a single input method, the combination of complementary interaction modalities enables users to complete inspection tasks more naturally and efficiently. Consequently, the developed system represents a practical contribution to the field of Human-Computer Interaction by illustrating the feasibility of integrating gesture recognition, voice recognition, and projection mapping into a unified real-time inspection platform.

The constructed system has various limits concerning the hardware employed during experimentation, predefined voice commands, and environmental conditions, however these constraints do not make the suggested approach less viable. Overall, the objectives of this study were successfully achieved, and the developed multimodal projection mapping-based 3D object inspection system provides a functional foundation for future research and more advanced interactive visualization applications.

REFERENCES

- [1] Carfi, A., & Mastrogiovanni, F. (2022). Human–machine interaction through gesture recognition: State of the art and future perspectives. *Robotics and Autonomous Systems*, 152, 104069. <https://doi.org/10.1016/j.robot.2022.104069>
- [2] Dritsas, E. (2025). Multimodal interaction systems and natural human–computer communication approaches. *International Journal of Human–Computer Interaction*, 41(2), 155–172. <https://doi.org/10.1080/10447318.2024.2387612>
- [3] Mohamed, R. A. (2024). Real-time hand gesture recognition systems for interactive environments: Challenges and optimization techniques. *Journal of Real-Time Image Processing*, 21(3), 45–61. <https://doi.org/10.1007/s11554-024-01372-1>
- [4] Printezis, D. (2026). Projection-based visualization and spatial augmented reality systems for immersive interaction environments. *Computers & Graphics*, 118, 103–118. <https://doi.org/10.1016/j.cag.2025.11.004>
- [5] Reski, A. (2024). Gesture interaction techniques in immersive three-dimensional visualization environments. *Virtual Reality & Intelligent Hardware*, 6(1), 22–37. <https://doi.org/10.1016/j.vrih.2023.11.002>
- [6] Survey on hand gesture recognition from visual input. (2025). *ACM Computing Surveys*, 58(1), 1–34. <https://doi.org/10.1145/3701128>
- [7] Wang, W., Li, R., Chen, Y., & Zhang, H. (2022). Gesture-based human–robot interaction: A systematic review. *Robotics and Computer-Integrated Manufacturing*, 75, 102312. <https://doi.org/10.1016/j.rcim.2022.102312>
- [8] Dritsas, E. (2025). Multimodal interaction systems in human–computer interaction environments. *Journal of Contemporary Educational Research*, 9(3), 12–20. <https://doi.org/10.26689/jcer.v9i3.494>
- [9] Al Farid, F., Hashim, R., & Ahmed, A. (2022). Vision-based hand gesture recognition methods in human–computer interaction: A review. *Journal of Imaging*, 8(6), 153. <https://doi.org/10.3390/jimaging8060153>
- [10] IEEE. (2021). Real-time hand gesture recognition for human–computer interaction systems. *IEEE Xplore*. <https://ieeexplore.ieee.org/document/9641587>
- [11] IEEE. (2024). Advanced gesture recognition methodologies for immersive interaction systems. *IEEE Xplore*. <https://ieeexplore.ieee.org/document/10669579>

- [12] NeurIPS. (2025). Benchmark datasets for real-time gesture recognition and multimodal interaction systems. *Advances in Neural Information Processing Systems*. https://proceedings.neurips.cc/paper_files/paper/2025/file/f92532079ab50a5db4109ca8f0cb0849-Paper-Datasets_and_Benchmarks_Track.pdf
- [13] Mahmood, H. S., & Abdulazeez, A. M. (2024). Voice recognition based on machine learning classification algorithms: A review. *Indonesian Journal of Computer Science*, 13(3), 4414–4431. <https://doi.org/10.33022/ijcs.v13i3.4110>
- [14] Sultana, N., Hossain, M. S., & Rahman, M. M. (2021). Voice recognition system for human–computer interaction using machine learning approaches. In *2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS)* (pp. 1192–1197). IEEE. <https://doi.org/10.1109/ICAIS50930.2021.9395936>
- [15] Bertolotti, I. C. (2025). Rethinking Modbus-UDP for real-time IIoT systems. *Future Internet*, 17(8), 356. <https://doi.org/10.3390/fi17080356>
- [16] Isaksson, O. (2025). *Development of bridge between C++ and Unity environment to visualize simulations* (Master’s thesis, Lappeenranta–Lahti University of Technology LUT). <https://lutpub.lut.fi/handle/10024/170838>
- [17] Shu, N., & Anderson, D. V. (2024). *Audiosockets: A Python socket package for real-time audio processing*. arXiv. <https://arxiv.org/abs/2403.09789>
- [18] Hellmuth, C.-P., Bachinski, M., & Müller, J. (2021). Interaction techniques for 3D-positioning objects in mobile augmented reality. In *Proceedings of the 2021 International Conference on Multimodal Interaction* (pp. 605–611). ACM. <https://doi.org/10.1145/3462244.3479945>
- [19] Printezis, P. (2026). Spatial augmented reality storytelling in arts and culture. *Heritage*, 9(1), 20. <https://doi.org/10.3390/heritage9010020>
- [20] Sun, J. (2025). The integration of virtual reality and augmented reality technology in immersive media art. *Smart Agricultural Technology and Sustainable Cities*, 2, 200377. <https://doi.org/10.1016/j.sasc.2025.200377> (mdpi.com)
- [21] Pan, Y. (2025). Design and development of virtual reality teaching systems based on Unity Engine. In *Proceedings of the 2025 International Conference on Interactive Digital Media*. ACM. <https://dl.acm.org/doi/10.1145/3765325.3765379>

- [22] Adetiba, E., John, T., Akinrinmade, A., Moninuola, F., Akintade, O., & Badejo, J. (2021). Evolution of artificial intelligence languages: A systematic literature review. *arXiv*. <https://arxiv.org/abs/2101.11501> (arxiv.org)
- [23] Al Farid, F., Hashim, R., & Ahmed, A. (2022). Vision-based hand gesture recognition methods in human–computer interaction: A review. *Journal of Imaging*, 8(6), 153. <https://doi.org/10.3390/jimaging8060153>
- [24] Yao, Z., Wu, D., Wang, X., Zhang, B., Yu, F., Yang, C., Peng, Z., Chen, X., Xie, L., & Lei, X. (2021). *WeNet: Production oriented streaming and non-streaming end-to-end speech recognition toolkit*. arXiv. <https://arxiv.org/abs/2102.01547>(arXiv)
- [25] Bertolotti, I. C. (2025). Rethinking Modbus-UDP for real-time IIoT systems. *Future Internet*, 17(8), 356. <https://doi.org/10.3390/fi17080356>
- [26] Wang, H., Guo, B., Zeng, Y., Ding, Y., Qiu, C., Zhang, Y., Yao, L., & Yu, Z. (2022). *Enabling harmonious human-machine interaction with visual-context augmented dialogue system: A review*. arXiv. <https://arxiv.org/pdf/2207.00782>
- [27] Wang, Z., Rao, M., Ye, S., Song, W., & Lu, F. (2025). Towards spatial computing: Recent advances in multimodal natural interaction for XR headsets. arXiv. <https://arxiv.org/abs/2502.07598>
- [28] Zhang, Y., Orthmann, B., Ji, S., Welle, M., Van Haastregt, J., & Kragic, D. (2025). Multimodal “Puppeteer”: An exploration of robot teleoperation via virtual counterpart with LLM-driven voice and gesture interaction in augmented reality. arXiv. <https://arxiv.org/abs/2506.13189>
- [29] Zhou, X., Williams, A. S., & Ortega, F. R. (2022). Eliciting multimodal gesture+speech interactions in a multi-object augmented reality environment. arXiv. <https://arxiv.org/abs/2207.12566>
- [30] Mohamed, A. S., Hassan, N. F., & Jamil, A. S. (2024). *Real-time hand gesture recognition: A comprehensive review of techniques, applications, and challenges*. *Cybernetics and Information Technologies*, 24(3), 163–184. <https://doi.org/10.2478/cait-2024-0031>
- [31] Marques, P., Silva, R., & Costa, J. (2025). *Real-time gesture-based hand landmark detection for interactive systems*. *Electronics*, 14(4), 704. <https://doi.org/10.3390/electronics14040704>
- [32] Biswas, S., & Mishra, D. (2023). *MediaPipe with LSTM architecture for real-time hand gesture recognition*. *Proceedings of CVIP 2023*.

- [33] Al Farid, F., Hashim, R., & Ahmed, A. (2022). Vision-based hand gesture recognition methods in human–computer interaction: A review. *Journal of Imaging*, 8(6), 153. <https://doi.org/10.3390/jimaging8060153>
- [34] Marques, P., Silva, R., & Costa, J. (2025). Real-time gesture-based hand landmark detection for interactive systems. *Electronics*, 14(4), 704. <https://doi.org/10.3390/electronics14040704>
- [35] Harihar, D. (2025). Voice-based user interface for hands-free data entry and automation at workplaces. *MethodsX*, 15, 103596. <https://doi.org/10.1016/j.mex.2025.103596>
- [36] Shanmuganathan, V. (2025). VoxBot: Enhancing human–robot interaction through speech recognition and voice-driven navigation. Premier Science Publications. <https://premierscience.com/pjs-25-1178/>
- [37] Hasyim, H., & Bakri, M. (2024). Advancements in human–computer interaction: A review of recent research. *Advances: Jurnal Ekonomi & Bisnis*, 2(4), 213–227. <https://doi.org/10.60079/ajeb.v2i4.327>
- [38] Liu, Y. (2025). A new human–computer interaction paradigm: Agent interaction model based on large models and its prospects. *Virtual Reality & Intelligent Hardware*, 7(3). <https://doi.org/10.1016/j.vrih.2025.04.001>

APPENDIX

ObjectControl.cs

```
using UnityEngine;
using TMPro;

public class ObjectController : MonoBehaviour
{
    public float rotationSpeed = 20f;
    public float zoomStep = 5.0f;

    public Transform robotObject;

    public Camera mainCamera;
    public TMP_Text statusText;
    public TMP_Text infoText;

    public float focusDistance = 2.0f;
    public float focusHeight = 0.3f;
    public float cameraMoveSpeed = 3f;

    public float cameraSide = 1f;

    public Transform headTarget;
    public Transform leftArmTarget;
    public Transform rightArmTarget;
    public Transform leftLegTarget;
    public Transform rightLegTarget;

    public GameObject headLabel;
    public GameObject leftArmLabel;
    public GameObject rightArmLabel;
    public GameObject leftLegLabel;
    public GameObject rightLegLabel;
```

```

public Renderer headRenderer;
public Renderer leftArmRenderer;
public Renderer rightArmRenderer;
public Renderer leftLegRenderer;
public Renderer rightLegRenderer;

public Material highlightMaterial;

private Material originalHeadMaterial;
private Material originalLeftArmMaterial;
private Material originalRightArmMaterial;
private Material originalLeftLegMaterial;
private Material originalRightLegMaterial;

private Vector3 targetCameraPosition;
private Quaternion targetCameraRotation;
private bool isFocusing = false;

private Quaternion originalRobotRotation;
private Vector3 originalRobotScale;

private Vector3 originalCameraPosition;
private Quaternion originalCameraRotation;

private SocketReceiver socketReceiver;

void Start()
{
    socketReceiver = GetComponent<SocketReceiver>();

    if (robotObject == null)
        robotObject = transform;

    originalRobotRotation = robotObject.rotation;
    originalRobotScale = robotObject.localScale;

    if (mainCamera != null)

```

```

{
    originalCameraPosition = mainCamera.transform.position;
    originalCameraRotation = mainCamera.transform.rotation;
}

if (headRenderer != null)
    originalHeadMaterial = headRenderer.material;

if (leftArmRenderer != null)
    originalLeftArmMaterial = leftArmRenderer.material;

if (rightArmRenderer != null)
    originalRightArmMaterial = rightArmRenderer.material;

if (leftLegRenderer != null)
    originalLeftLegMaterial = leftLegRenderer.material;

if (rightLegRenderer != null)
    originalRightLegMaterial = rightLegRenderer.material;

HideAllLabels();
ClearHighlights();

SetStatus("Waiting for Command");
HideInfo();
}

void Update()
{
    if (isFocusing && mainCamera != null)
    {
        mainCamera.transform.position = Vector3.Lerp(
            mainCamera.transform.position,
            targetCameraPosition,
            Time.deltaTime * cameraMoveSpeed
        );
    }
}

```

```

mainCamera.transform.rotation = Quaternion.Lerp(
    mainCamera.transform.rotation,
    targetCameraRotation,
    Time.deltaTime * cameraMoveSpeed
);

if (Vector3.Distance(mainCamera.transform.position, targetCameraPosition) < 0.05f)
    isFocusing = false;
}

if (socketReceiver == null || string.IsNullOrEmpty(socketReceiver.lastCommand))
    return;

string cmd = socketReceiver.lastCommand;

if (cmd == "ROTATE")
{
    SetStatus("Rotating Object");

    if (robotObject != null)
        robotObject.Rotate(Vector3.up, rotationSpeed * Time.deltaTime, Space.World);
}

else if (cmd == "ZOOM")
{
    SetStatus("Zooming Object");

    if (mainCamera != null && robotObject != null)
    {
        Vector3 robotCenter = robotObject.position + new Vector3(0, 1.0f, 0);

        mainCamera.transform.position = Vector3.MoveTowards(
            mainCamera.transform.position,
            robotCenter,
            zoomStep * Time.deltaTime
        );
    }
}

```

```

        mainCamera.transform.LookAt(robotCenter);
    }
}

else if (cmd == "RESET")
{
    SetStatus("Waiting for Command");
    HideInfo();

    if (robotObject != null)
    {
        robotObject.rotation = originalRobotRotation;
        robotObject.localScale = originalRobotScale;
    }

    if (mainCamera != null)
    {
        mainCamera.transform.position = originalCameraPosition;
        mainCamera.transform.rotation = originalCameraRotation;
    }

    isFocusing = false;

    HideAllLabels();
    ClearHighlights();

    socketReceiver.lastCommand = "";
}

else if (cmd == "HEAD")
{
    SetStatus("Inspecting Head");
    ShowInfo("Head", "Upper robot component used for visual identification and inspection focus.");

    FocusOnPart(headTarget);
    ShowOnlyLabel(headLabel);
    HighlightOnly(headRenderer);
}

```

```

    socketReceiver.lastCommand = "";
}

else if (cmd == "LEFT_ARM" || cmd == "ARM")
{
    SetStatus("Inspecting Left Arm");
    ShowInfo("Left Arm", "Robot side component used for manipulation and movement representation.");

    FocusOnPart(leftArmTarget);
    ShowOnlyLabel(leftArmLabel);
    HighlightOnly(leftArmRenderer);

    socketReceiver.lastCommand = "";
}

else if (cmd == "RIGHT_ARM")
{
    SetStatus("Inspecting Right Arm");
    ShowInfo("Right Arm", "Robot side component used for manipulation and movement representation.");

    FocusOnPart(rightArmTarget);
    ShowOnlyLabel(rightArmLabel);
    HighlightOnly(rightArmRenderer);

    socketReceiver.lastCommand = "";
}

else if (cmd == "LEFT_LEG" || cmd == "LEG")
{
    SetStatus("Inspecting Left Leg");
    ShowInfo("Left Leg", "Lower robot component used for support and locomotion representation.");

    FocusOnPart(leftLegTarget);
    ShowOnlyLabel(leftLegLabel);
    HighlightOnly(leftLegRenderer);
}

```

```

        socketReceiver.lastCommand = "";
    }

    else if (cmd == "RIGHT_LEG")
    {
        SetStatus("Inspecting Right Leg");
        ShowInfo("Right Leg", "Lower robot component used for support and locomotion representation.");

        FocusOnPart(rightLegTarget);
        ShowOnlyLabel(rightLegLabel);
        HighlightOnly(rightLegRenderer);

        socketReceiver.lastCommand = "";
    }
}

void SetStatus(string message)
{
    if (statusText != null)
        statusText.text = "Status: " + message;
}

void ShowInfo(string title, string description)
{
    if (infoText != null)
    {
        infoText.gameObject.SetActive(true);
        infoText.text = title + "\n\n" + description;
    }
}

void HideInfo()
{
    if (infoText != null)
        infoText.gameObject.SetActive(false);
}

```

```

void FocusOnPart(Transform target)
{
    if (target == null || mainCamera == null)
        return;

    targetCameraPosition = target.position + new Vector3(
        0,
        focusHeight,
        focusDistance * cameraSide
    );

    Vector3 direction = target.position - targetCameraPosition;
    targetCameraRotation = Quaternion.LookRotation(direction);

    isFocusing = true;
}

void HideAllLabels()
{
    if (headLabel != null) headLabel.SetActive(false);
    if (leftArmLabel != null) leftArmLabel.SetActive(false);
    if (rightArmLabel != null) rightArmLabel.SetActive(false);
    if (leftLegLabel != null) leftLegLabel.SetActive(false);
    if (rightLegLabel != null) rightLegLabel.SetActive(false);
}

void ClearHighlights()
{
    if (headRenderer != null && originalHeadMaterial != null)
        headRenderer.material = originalHeadMaterial;

    if (leftArmRenderer != null && originalLeftArmMaterial != null)
        leftArmRenderer.material = originalLeftArmMaterial;

    if (rightArmRenderer != null && originalRightArmMaterial != null)
        rightArmRenderer.material = originalRightArmMaterial;
}

```

```

        if (leftLegRenderer != null && originalLeftLegMaterial != null)
            leftLegRenderer.material = originalLeftLegMaterial;

        if (rightLegRenderer != null && originalRightLegMaterial != null)
            rightLegRenderer.material = originalRightLegMaterial;
    }

    void ShowOnlyLabel(GameObject label)
    {
        HideAllLabels();

        if (label != null)
            label.SetActive(true);
    }

    void HighlightOnly(Renderer selectedRenderer)
    {
        ClearHighlights();

        if (selectedRenderer != null && highlightMaterial != null)
            selectedRenderer.material = highlightMaterial;
    }
}

```

Gesture.py

```

import cv2
import mediapipe as mp
import math
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_address = ("127.0.0.1", 5052)

mp_hands = mp.solutions.hands

```

```

mp_draw = mp.solutions.drawing_utils

hands = mp_hands.Hands(
    max_num_hands=1,
    min_detection_confidence=0.6,
    min_tracking_confidence=0.6
)

cap = cv2.VideoCapture(0)

reset_sent = False
last_command = ""

def distance(p1, p2):
    return math.sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2)

def is_open(hand, tip, pip):
    return hand.landmark[tip].y < hand.landmark[pip].y

while True:
    success, img = cap.read()
    if not success:
        continue

    img = cv2.flip(img, 1)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    results = hands.process(img_rgb)

    label = "NO HAND"
    command = ""

    if results.multi_hand_landmarks:
        hand = results.multi_hand_landmarks[0]
        mp_draw.draw_landmarks(img, hand, mp_hands.HAND_CONNECTIONS)

        thumb = hand.landmark[4]
        index = hand.landmark[8]

```

```
pinch = distance(thumb, index)
```

```
open_hand = (  
    is_open(hand, 8, 6) and  
    is_open(hand, 12, 10) and  
    is_open(hand, 16, 14) and  
    is_open(hand, 20, 18)  
)
```

```
fist = (  
    not is_open(hand, 8, 6) and  
    not is_open(hand, 12, 10) and  
    not is_open(hand, 16, 14) and  
    not is_open(hand, 20, 18)  
)
```

```
if pinch < 0.05:  
    label = "ZOOM"  
    command = "ZOOM"  
    reset_sent = False
```

```
elif open_hand:  
    label = "ROTATE"  
    command = "ROTATE"  
    reset_sent = False
```

```
elif fist:  
    label = "RESET"  
    if not reset_sent:  
        command = "RESET"  
        reset_sent = True
```

```
else:  
    reset_sent = False
```

```
else:  
    reset_sent = False
```

```

if command and command != last_command:
    sock.sendto(command.encode(), server_address)
    print("Sent:", command)
    last_command = command

if not command:
    last_command = ""

cv2.putText(img, label, (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

cv2.imshow("Gesture Control", img)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

Voice.py

```

import speech_recognition as sr
import socket

# Socket setup
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_address = ("127.0.0.1", 5052)

recognizer = sr.Recognizer()
mic = sr.Microphone()

recognizer.energy_threshold = 300
recognizer.pause_threshold = 0.6
recognizer.dynamic_energy_threshold = True

```

```
def send_command(command):  
    sock.sendto(command.encode(), server_address)  
    print("Sent:", command)
```

```
def detect_command(text):  
    text = text.lower().strip()  
  
    # Gesture-related voice commands  
    if "spin" in text or "rotate" in text:  
        return "ROTATE"
```

```
    if "zoom" in text or "closer" in text:  
        return "ZOOM"
```

```
    if "reset" in text or "restart" in text:  
        return "RESET"
```

```
    # Component commands  
    if "head" in text:  
        return "HEAD"
```

```
    if (  
        "left arm" in text  
        or "left hand" in text  
        or "left-hand" in text  
    ):  
        return "LEFT_ARM"
```

```
    if (  
        "right arm" in text  
        or "right hand" in text  
        or "right-hand" in text  
        or "brighton" in text  
    ):
```

```

    return "RIGHT_ARM"

if "left leg" in text:
    return "LEFT_LEG"

if "right leg" in text:
    return "RIGHT_LEG"

# General fallback commands
if "arm" in text or "hand" in text:
    return "ARM"

if "leg" in text:
    return "LEG"

return None

print("Voice control started...")
print("Say: head, left arm, right arm, left leg, right leg, rotate, zoom, or reset")

while True:
    with mic as source:
        recognizer.adjust_for_ambient_noise(source, duration=0.5)
        print("\nListening...")
        audio = recognizer.listen(source, phrase_time_limit=3)

    try:
        text = recognizer.recognize_google(audio).lower()
        print("You said:", text)

        command = detect_command(text)

        if command is not None:
            send_command(command)
        else:
            print("No matching command")

```

```
except sr.UnknownValueError:  
    print("Could not understand")
```

```
except sr.RequestError as e:  
    print("API error:", e)
```